M362 Unit 7
UNDERGRADUATE COMPUTING

Developing concurrent distributed systems

The business tier

Unit 7

This publication forms part of an Open University course M362 *Developing concurrent distributed systems*. Details of this and other Open University courses can be obtained from the Student Registration and Enquiry Service, The Open University, PO Box 197, Milton Keynes MK7 6BJ, United Kingdom: tel. +44 (0)845 300 60 90, email general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at http://www.open.ac.uk where you can learn more about the wide range of courses and packs offered at all levels by The Open University.

To purchase a selection of Open University course materials visit http://www.ouw.co.uk, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858793; fax +44 (0)1908 858787; email ouw-customer-services@open.ac.uk

The Open University
Walton Hall
Milton Keynes
MK7 6AA

First published 2008.

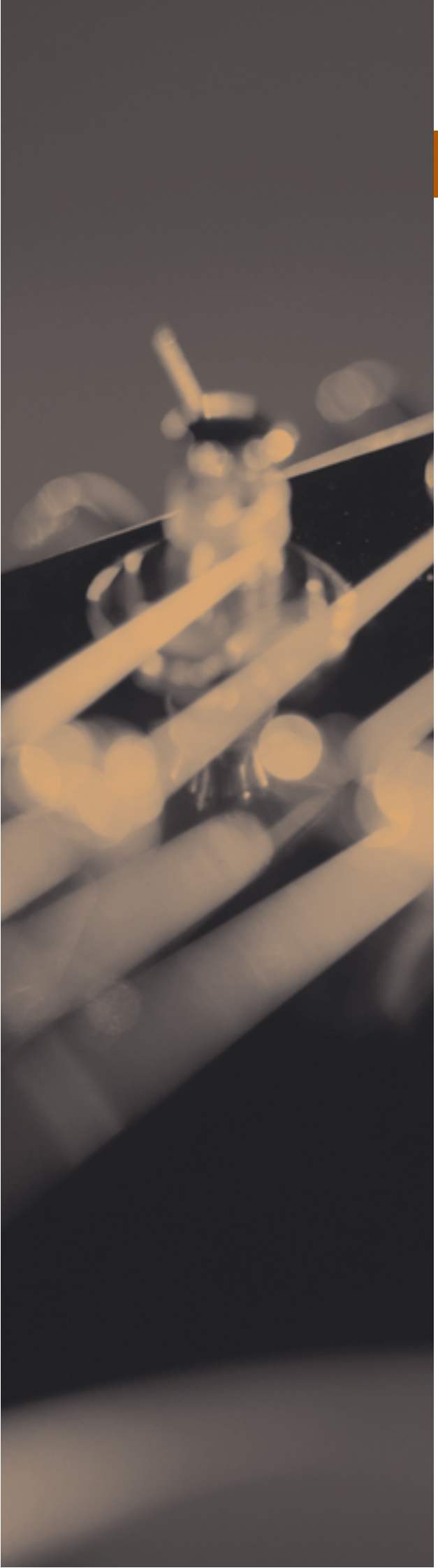
Edited and designed by The Open University.

Typeset by SR Nova Pvt. Ltd, Bangalore, India.

Printed and bound in the United Kingdom by Martins the Printers, Berwick-upon-Tweed.

ISBN 978 0 7492 1596 5

1.1



CONTENTS

M362 COURSE TEAM

Affiliated to The Open University unless otherwise stated.

Chair, author and academic editor

Janet van der Linden

Authors

Anton Dil

Brendan Quinn

Michel Wermelinger

Critical readers and testers

Henryk Krajinski

Barbara Segal

Mark Thomas

Richard Walker

Yijun Yu

External assessor

Aad van Moorsel, Newcastle University

Software development

Ivan Dunn, Consultant

Course management

Linda Landsberg

Carrie Lewis

Barbara Poniatowska

Julia White

Media development staff

Ian Blackham, Editor

Sarah Gamman, Contracts Executive

Jennifer Harding, Editor

Phillip Howe, Media Assistant

Martin Keeling, Media Assistant

Callum Lester, Software Developer

Andy Seddon, Media Project Manager

Sue Stavert, Technical Testing Team

Andrew Whitehead, Designer and Graphic Artist

Thanks are due to the Desktop Publishing Unit of the Faculty of Mathematics, Computing and Technology.

1 Introduction

As we have seen in *Unit 5*, Subsection 3.4, in a three-tier architecture the business tier is the place where most of the enterprise application's logic is defined, i.e. where the workflow of the business activities is encoded. This tier has, on the one hand, to use the storage and transaction services provided by the database tier we looked at in *Unit 6*, and, on the other hand, to provide services to the web tier, which is the subject of the next unit.

We are thus dealing with more than one tier from this stage on, and different tiers are potentially located on different machines. We must therefore start by understanding how Java implements the generic distributed object model presented in *Unit 5*, Subsection 7.2. For such a model to work it must be possible to call the methods of remote objects and to send objects across the network as arguments of those method calls. Although many of the details are hidden by the Java Enterprise Edition (Java EE) infrastructure, it is important to understand how the distributed object model is implemented, because it raises some recurrent issues, namely the way in which distributed objects (or services in general) are named and located.

We then move on to the main issues of this unit: how does the business tier interface with the resource tier, and how does it define services that encapsulate the business activities? We will see that Java EE advocates the separation of the business data and the business logic into two kinds of class. The first kind of class hides the JDBC calls we have seen in *Unit 6*, Section 7, and directly represents, in an object-oriented way, the data contained in the relational tables of the resource tier. The second kind of class contains the code to process the data in response to requests coming from the web or client tiers.

1.1 The aims of this unit

In this unit you will study:

- distributed objects, and the communication between them that enables objects running on one machine to locate objects running on other machines and call their methods (Section 2);
- how to address some of the naming issues for distributed systems (Section 3);
- how to program queries and changes to a relational database in an object-oriented way (Section 4);
- the Java EE classes that make it possible to program business logic separately from data access (Section 5).

This unit involves reading and practical activities that will give you experience in programming the business tier of simple applications.

2 Remote Method Invocation

Bear in mind that RMI is specific to Java; see later in *Unit 11* for a language-independent approach to distributed objects.

Unit 5, Subsection 7.2, introduced the main concepts of distributed objects, and in this section we look in some detail at **Remote Method Invocation** (**RMI**), Java's implementation of those concepts. Simply put, RMI is an object-oriented counterpart to the remote procedure call (RPC) mechanism, which we met in *Unit 5*, Subsection 7.1.

Our interest in RMI is three-fold.

- It provides a concrete example of the distributed object paradigm.
- It raises the naming, locating and binding problems that are at the very core of distributed systems.
- It is a technology that can be used by itself, outside the Java EE context, in many Java applications.

2.1 The architecture

As we saw in *Unit 4*, Subsection 2.2, each Java Virtual Machine (JVM) process has a memory area, called the heap, where all objects are stored. An object reference is simply a memory pointer to the heap location where the object is stored. It is therefore easy for objects within the same JVM to hold references to each other and in that way to be able to call each other's methods.

However, consider a client–server system in a distributed setting: it comprises a client application running in a JVM on the client host, and a server application running in a different JVM on a different host machine. Suppose we have an object on the client side, which we will call the client object, and an object on the server side, which we will call the server object. In such a scenario, how can the client object obtain a reference to the server object, so that the client object can call methods on the server object? Note that the server object cannot simply make its memory location publicly available because the operating system protects the memory of each process from external access.

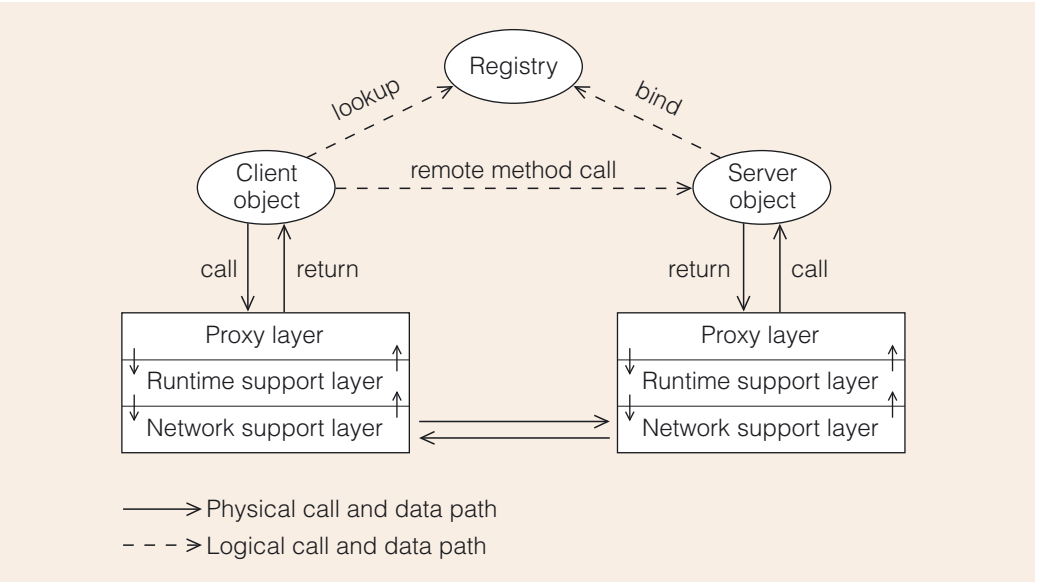


Figure 1 A generic architecture for remote method calls (adapted from Liu, 2004, p. 206)

A generic layered architecture for solving this problem is presented in Figure 1. The steps that take place are:

The concept of layered architecture was introduced in *Unit 5*, Subsection 3.2.

1 The server application binds the server object to a symbolic name in a **registry**, a naming service that associates objects to unique names. The location of the registry and the symbolic name are known to client and server.
2 The client object looks up the symbolic name in the registry and obtains a reference to the server object.
3 The client object uses that reference to make a remote method call.

The remote method call is implemented as a series of local method calls, both on the client side and on the server side as Figure 1 shows, together with transmission over the network of the necessary arguments and return value. Each proxy layer contains a **proxy object**, i.e. a representative of the server object. When the registry returns a reference to the server object in step 2, it is in fact passing a proxy object to the client. The proxy objects have the same methods as the server object but they do not do any actual work, they just pass the method call onwards and any return value backwards.

The concept of a proxy as an intermediary was introduced in *Unit 5*, Subsection 2.3, but in a different context.

The proxy layer uses the runtime support layer to marshal and unmarshal the method's arguments and the return value. The runtime support is responsible for doing the internal 'reference bookkeeping' that ensures that client-side and server-side objects and proxies are appropriately 'paired' with each other. Otherwise a call from the client might end up being received by the wrong server object. The runtime support layer uses the existing network support layer to actually send all the necessary information around via the low-level network communication protocols, such as sockets and TCP/IP packets.

Data marshalling was explained in Unit 5, Subsection 5.4.

Exercise 1

Which data is marshalled (or unmarshalled) by the runtime support layer and where does this take place?

Discussion .....

The method arguments are marshalled on the client side and unmarshalled on the server side. The return value is marshalled on the server side and unmarshalled on the client side.

The generic approach to remote method calls we have just outlined is similar to its procedural equivalent, the remote procedure call (RPC) mechanism.

SAQ 1

What is similar and what is different between the implementation of remote method calls and remote procedure calls as presented in *Unit 5*, Subsection 7.1?

ANSWER.....

Both remote method calls and RPC are blocking forms of communication: the caller cannot proceed with its execution until the called method or procedure has returned. Moreover, as with RPC, the method call has to go through various layers that abstract away data marshalling and the low-level network protocols. For example, the runtime support layer in Figure 1 corresponds to the RPC service layer. The discussion on what can go wrong in an RPC call (e.g. network disruption) also applies in the distributed object context and has therefore been omitted from the presentation above.

The main difference is that in RPC a procedure is called directly, while in an object-oriented approach a method is called through a particular object that was created at runtime. The implementation of remote method calls therefore requires a registry and a unique name to look up the object reference.

Java's RMI, which this section will consider, follows the above architecture. Note that in the context of RMI, 'local' means 'within the same JVM' and 'remote' means 'across different JVMs'. For example, a **remote method** is a method that can be called from outside the JVM where it is executing, and a **remote object** is an object with at least one remote method. In a typical client–server scenario, the server object is a remote object, but the client object is not.

The RMI registry service, the client application and the server application are thus possibly, but not necessarily, executing on three different JVMs, and on three different machines. In fact, for security reasons, the registry implementation provided by Sun Microsystems *must* run on the same host as the server, to ensure that clients cannot change the registry, only look it up.

Note that skeletons have now been replaced by another technology, but we will use them in this section because they facilitate the explanation of how RMI works.

In RMI the proxy layer is called the stub/skeleton layer, the **stub object** being the client-side proxy object and the skeleton object being the server-side proxy object. However, the exact details of the proxy layer do not concern us as proxies are automatically created and managed by the RMI infrastructure. All the developer has to do is to **export** the remote object. This will make the RMI runtime system create the corresponding stub and skeleton objects and start accepting incoming remote calls on that remote object (via the skeleton object).

After a remote object has been created and exported, it can be bound to a name in the registry. Conceptually, the name is bound to the remote object. Actually, the registry contains the corresponding stub object, i.e. the binding process basically puts a name–stub pair in the registry. When the client looks up a name in the registry, it gets back the associated stub object, on which it can now make local method calls: the stub object, together with the RMI and network infrastructure, will forward the call and its arguments to the skeleton object and pass any return value to the client. When the server JVM receives the call, it starts a new thread to execute it. This makes it possible for a client to call a remote method at any time. It also means that multiple clients may simultaneously call the same remote object's method, which therefore may have to be `synchronized` (see *Unit 4*, Section 4).

It should be noted that the mapping between names and objects is kept only while the registry runs; there is no persistent storage. Names have to be unique within a single registry but different registries can associate the same name with different objects.

Figure 2 shows the steps involved in setting up a remote method invocation. It shows a possible scenario involving two clients that wish to invoke methods on the same server object. The figure illustrates a major point: that there are multiple copies of stub objects for the same remote object. The initial copy is put by the server on the registry, when binding the remote object to a name. Each client obtains a further copy of the stub object when looking up the registry by name.

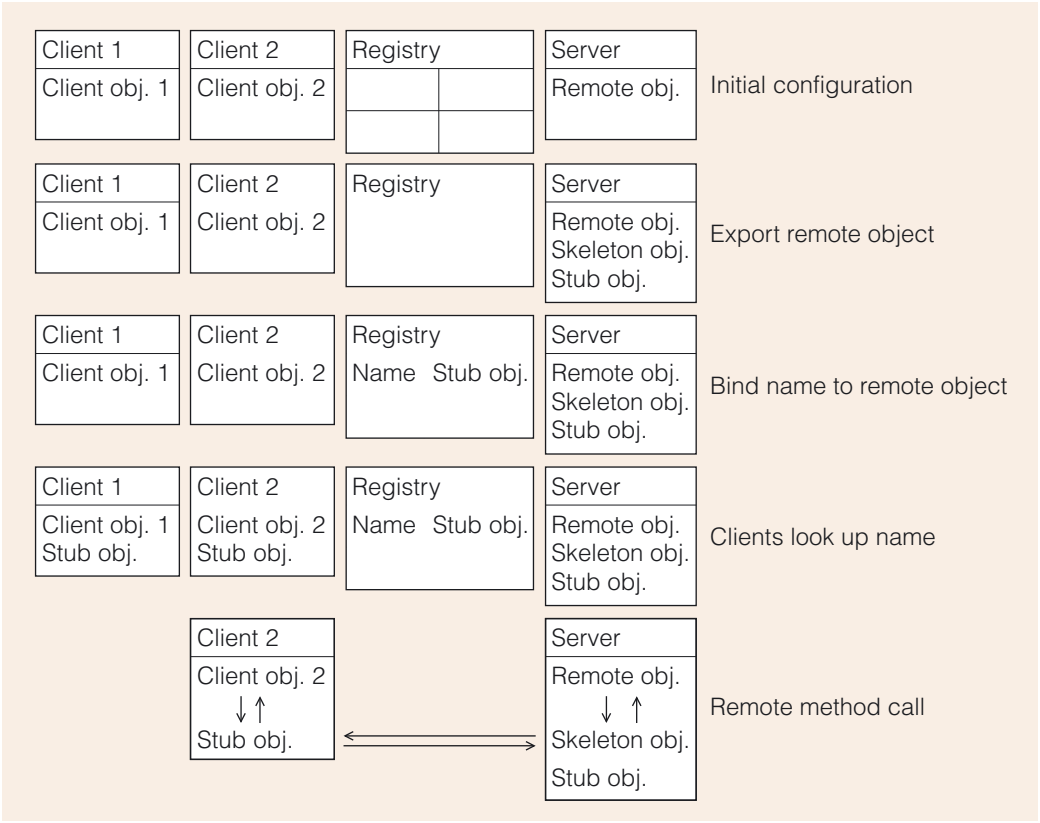


Figure 2 Steps involved in setting up a remote method invocation

To simplify the description, we have so far assumed everything goes to plan. But in a distributed setting, additional failure points are possible due to communication breakdown between the client and server hosts. Moreover, in the context of the RMI architecture, further errors may occur: the registry service may not be available or the name may not be found, etc. Java handles all this in the usual way: it raises exceptions which the developer must catch and deal with appropriately in the context of the application. We will see more details of this in the next section.

2.2 The Remote Method Invocation API

To illustrate how to program with RMI, we consider a concrete application, a computation engine, to which CPU-intensive tasks can be submitted, e.g. scientific calculations such as climate change modelling. The idea is to take advantage of distributed computing by having the computation engine reside on a powerful machine, with one or more fast processors and a large quantity of memory. The computation engine will be implemented by a class with one method for each calculation it provides.

To keep the example simple, we will assume that our computation engine makes just one method available, to calculate the factorial of a given positive integer n. Notice that the factorial function grows very quickly as n increases; e.g. the factorial of 10 is 3 628 800. We will assume that $n \leqslant 20$, so that $n!$ fits in a `long`.

The factorial of n is written $n!$ and is the product of all the integers from 1 to n.

The server-side code

The first step in developing the computation engine is to define it as a **remote interface**, which will specify the methods available to client objects. It is a remote interface because objects of classes implementing this interface will be accessed remotely, i.e. from outside the JVM they are residing in. This is indicated by extending the `java.rmi.Remote` interface. Moreover, due to the additional problems that can occur

when calling remote methods (e.g. communication failure) every remote method has to declare that it may throw a `java.rmi.RemoteException`.

```
package engine;

import java.rmi.Remote;
import java.rmi.RemoteException;

public interface ComputationEngine extends Remote
{
  public long factorial(int n) throws RemoteException;
}
```

The second step is to define a class that implements the above interface. In general, the remote object might include additional helper methods that cannot be directly called by clients, but there will be none for our simple example.

Unicast communication means one-to-one, point-to-point communication.

As explained in the previous section, a remote object must be exported so that it can accept client calls. The simplest way to guarantee that a remote object is properly created and exported is to extend the `java.rmi.server.UnicastRemoteObject` class. This class has a constructor without arguments that does all that is required. It will throw a `RemoteException` if something goes wrong during the creation and exporting process. The server object class constructors have only to make sure they call the `UnicastRemoteObject` constructor and either deal with any exceptions or pass them on.

Putting all this together, our computation engine implementation is the following.

```
package engine;

import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;

public class ComputationEngineImpl extends UnicastRemoteObject
                                   implements ComputationEngine
{
  public ComputationEngineImpl() throws RemoteException
  {
    super();
  }

  public long factorial(int n) throws RemoteException
  {
    long result = 1;

    for (int i = 2; i <= n; i++)
    {
      result = result * i;
    }
    return result;
  }
}
```

The third step is to create the server application. At some point, usually in the `main` method, the application will need to create a remote object and bind it to some name. We present the code and then explain it.

```
package engine;

import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;
import java.rmi.server.UnicastRemoteObject;

public class Main
{

  public static void main(String[] args)
  {
    ComputationEngine engine;
    Registry registry;

    try           // try to create registry
    {
      registry =
          LocateRegistry.createRegistry(Registry.REGISTRY_PORT);
      System.out.println("Registry created.");
    }
    catch(Exception e)
    {
      System.out.println("Registry not created: "
                         + e.getMessage());
      return;
    }

    try           // try to create remote object
    {
      engine = new ComputationEngineImpl();
      System.out.println("Engine created.");
    }
    catch(Exception e)
    {
      System.out.println("Engine not created: "
                         + e.getMessage());
      return;
    }

    try           // try to bind name in registry
    {
      registry.bind("computation_engine", engine);
      System.out.println("Engine registered.");
    }
    catch(Exception e)
    {
      System.out.println("Engine not registered: "
                         + e.getMessage());
      return;
    }
```

```
        try           // try to list names in the registry
        {
          System.out.println("Bound names: ");
          for (String s : registry.list())
          {
            System.out.println(s);
          }
        }
        catch(Exception e)
        {
          System.out.println("Registry listing error: "
                             + e.getMessage());
          return;
        }
      }
    }
```

First we create the registry using the static `createRegistry` method. The registry will run within the same JVM (and hence the same host) as the computation engine, and the method takes as argument the port on which the registry will wait for requests to bind objects and lookup names. We use the default registry port, which is given by the `REGISTRY_PORT` constant. We catch any exception thrown (e.g. because the given port is already in use), and return.

Then we create the remote object. Note that the declared type of the object is the remote interface to emphasise that this is how the object will be seen externally. We catch any exception thrown while creating and exporting the object, and return immediately.

Afterwards we bind the computation engine to the name `computation_engine` in the RMI registry. The registry's `bind` method will raise an exception if the registry could not be contacted for some reason, or if the name is already bound. The last exception cannot occur if we use the `rebind` method instead, which overrides any previously existing binding for the given name. There is also an `unbind(String name)` method which removes from the registry the corresponding name–stub pair. We call the `list` method, which returns an array with one string for each bound name, to confirm that the engine was registered in the registry.

SAQ 2

What are a remote object, a remote interface and a remote method? How can they be detected in the server-side code?

ANSWER

A remote object is an instance of a class that implements at least one remote interface. A remote interface is an interface that declares one or more remote methods. A remote method is a method that can be called from another JVM.

In the code one has to look for the following: a remote interface extends `Remote`, a remote method throws a `RemoteException` and a remote object's class usually extends `UnicastRemoteObject`.

It might seem that the server application terminates after it has registered the object. If that were so, by the time clients call the factorial method, the computation engine would no longer be running! So what is going on?

As we said before, once a remote object is exported, it waits for calls to its methods. These calls may come at any time. The application that exported the remote object can therefore not exit while the remote object exists. Only when the remote object is 'unexported' can the application terminate. The static method

```
boolean UnicastRemoteObject.unexportObject(Remote obj, boolean force)
```

will unexport the `obj` object and therefore any further remote calls to it will raise an exception. If the `force` Boolean is `false`, the object will be unexported only if there are no pending or currently executing calls to its methods. If the Boolean is `true`, the object will be forcibly unexported. The `unexportObject` method returns `true` only if `obj` was successfully unexported.

The client-side code

Having completed the server-side code, we can turn our attention to the client-side code. The developer has to find out – e.g. by consulting the documentation of the server application – which is the appropriate remote interface (in our example `ComputationEngine`) and the name under which the remote object will be bound (in our example `computation_engine`). In practice, the remote interface Java file may be available for download from the server, so that the client application can be compiled.

Once the remote interface is available, the client first obtains the `Registry` object (more details on this later), then calls the registry's `lookup` method to return the stub object associated with the `computation_engine` name in the registry, and finally reads a sequence of integers from the standard input, calling the factorial method on the stub object for each value. To read the integers we will use the utility class `Scanner`. This class (available since Java 1.5) is for reading input from a variety of sources, including the standard input stream `System.in` (usually the keyboard). Input is read in as a set of 'tokens', separated by space, tab or newline characters. The code is as follows.

```
package client;

import engine.ComputationEngine;
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;
import java.util.Scanner;

public class Main
{
  public static void main(String[] args)
  {
    ComputationEngine engine;
    Registry registry;

    try               // try to locate registry
    {
      registry = LocateRegistry.getRegistry();
      System.out.println("Registry stub created.");
    }
    catch(Exception e)
    {
      System.out.println("Registry stub not created: "
                         + e.getMessage());
      return;
    }
```

```
        try                // try to look up the remote object
        {
          engine =
            (ComputationEngine) registry.lookup("computation_engine");
          System.out.println("Engine found.");
        }
        catch(Exception e)
        {
          System.out.println("Engine not found: "
                             + e.getMessage());
          return;
        }

        try                // try to call the remote method
        {
          Scanner input = new Scanner(System.in);
          System.out.print("Please input the values, ");
          System.out.println("separated by spaces, tabs or newlines.");
          while (input.hasNext())
          {
            int n = input.nextInt();
            long f = engine.factorial(n);
            System.out.println(n + "! = " + f);
          }
        }
        catch(Exception e)
        {
          System.out.println("Factorial not computed: "
                             + e.getMessage());
        }
      }
    }
```

Note that we catch any exceptions that might be thrown – these may arise for a variety of reasons such as the registry not being contactable, there being no object registered under the given name, the next item in the input not being an integer, etc.

The most interesting part of the above code is how a registry object is obtained, so that the `lookup` method can be called. Just as the computation engine is a remote object that provides some methods (in our case, just `factorial`), the registry is also a remote object that provides a set of methods, including `bind`, `list` and `lookup`. It is a remote object because its methods can be called from code running in a JVM other than the one where the registry is running. In our scenario, the engine and registry are on the same JVM and therefore only the `lookup` method is called remotely.

As we have seen before, remote objects require stub objects that forward the calls and receive any return value. But how does a client obtain the stub object for an already existing registry? For 'normal' stub objects, clients get them via a unique known name. For `Registry` stub objects, clients get them by knowing on which host and port the registry is running. The static `LocateRegistry.getRegistry(String hostURL, int portNumber)` method returns a `Registry` stub object that will forward calls to the remote `Registry` object that was exported to the given port on the given machine. If the host is the same machine as the client is running on, and the port is the default RMI port, then the two parameters can be omitted. In our simplified example, we are assuming that client and engine will be run on the same host (your home machine), and therefore we used the no arguments version of `getRegistry`.

It is important to realise that `getRegistry` creates the stub object, but does not establish a connection to the corresponding remote `Registry` object. The connection is only made upon the first call on the stub object (typically a `lookup`). This means that `getRegistry` will not check if there is actually a registry service at the given machine and port, and the first call on the stub object may throw an exception. Therefore, the message in our code says only that the registry stub was created, not that the registry was found.

SAQ 3

We have previously said that the default registry implementation provided by the Java library requires the registry to run on the *same host* as the server application. Does this contradict the above sentence stating that the registry's services are *remote* methods?

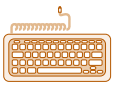
ANSWER.....

There is no contradiction: remote methods *may* be called from a different *JVM* not necessarily from a different *host*. To simplify the presentation of RMI, in this course we create the registry from within the server application, but there are ways to have the registry run on its own JVM.

The last point to make about our client code is that the call to the remote object is syntactically indistinguishable from a normal (i.e. local) method call:

```
long f = engine.factorial(n);
```

The reason is simple: it *is* a local call. The `engine` variable denotes a local stub object, not the remote object. This is an example of **location transparency** – the actual location of the remote object is unknown to the client. Only the stub object knows on which JVM the remote object is located; the client only knows where the registry is in order to look up the stub objects by name.



Activity 7.1
Factorial engine.

2.3 Serialisation and security

Conceptually, the stub object is the **remote reference** to the remote object. The remote object never leaves the JVM where it was created, but there will be one copy of the stub object in each JVM that communicates with the remote object. Whenever a client looks up a name, the registry returns a copy of the stub object it has stored. This means that the stub object is marshalled from the registry JVM (which may, or may not be, the same as the server JVM) to the client JVM. In Java, the marshalling of objects is called **object serialisation**. This is *not* the same as the serialisation we met in the context of threads (*Unit 4*) and transactions (*Unit 6*). In those cases, the question was how to ensure that *concurrent* operations could have the same effect as a *sequence* of operations. In the context of marshalling, serialisation simply means that the objects have to be 'flattened' (see *Unit 5*, Figure 16) into a serial stream of data which can be sent – possibly over a network – across JVM processes and rebuilt (deserialised) into *a copy of the object* by the receiver. If an object refers to another object, the serialisation process will also marshal the referred object. Only objects that have been explicitly defined as being serialisable will be serialised when passed across JVMs. We will see later how to declare a class of objects as being serialisable. Not all classes can have their objects serialised but those few exceptions do not concern us.

In the context of RMI, serialisation does not just take place when the stub object is returned by a lookup operation on the registry. It also occurs when the arguments or return value of the remote method are objects, rather than primitive data (integers, Booleans, etc.). This implies three things.

1 When defining the classes of those objects, they must be declared as serialisable.
2 If the arguments are (in)directly referring to many objects stored in the client JVM, all those objects will also be copied to the server, leading to a large communication overhead.
3 Since the server code is working on copies, and not the original objects stored on the client, changes made to the arguments by the remote object method will not be reflected on the client side.

Serialising objects marshals only their data. The method code is common to all objects of the same class and is therefore stored together with the class information (e.g. static variables). This means that if the client passes to the remote method an argument that is an instance of a class unknown to the server, the server will have to load the corresponding class at runtime. For that to be possible, the client has to tell the server where the class's code can be found. In the context of RMI, the client has to set the property `java.rmi.server.codebase` to a list of folders or URL addresses where the server can find the classes of the objects the client will send. The activity for the next subsection will show an example.

The runtime class loader that is part of a JVM was mentioned in *Unit 4*, Subsection 2.2.

Object serialisation and class loading at runtime make it possible for clients to send to servers (and, of course, vice versa) instances of classes the server has never seen before, and allow the server to execute methods on those instances. This raises the possibility that the code downloaded by the server might be malicious or buggy and cause many problems if it is granted full access to the server machine's resources. Such problems can be prevented by using **security managers**. A security manager checks that the operations to be performed conform to a **security policy** defined by the user. For example, a common policy is to prohibit downloaded code from reading from, or writing to, any file. Security is a big topic in its own right, and will be dealt with in detail in *Unit 10*. For the purposes of this unit, it is enough to say that without a security manager, a JVM can only dynamically load code that can be found in the class path, as was the case in Activity 7.1.

Prior to Java 1.5, the client also required a security manager and policy even if the server's remote methods did not return any object of a class that was not already known to the client. The reason for this was that the client needed to download from the server the class of the stub object returned by the registry. In Java 1.5, the stub class is generated at runtime by the RMI infrastructure on the client side and therefore no code downloading has to take place for that purpose.

RMI versus programming with sockets directly

Like many other technologies, RMI is based on the notion of layers that shield the developer from lower-level issues, providing a higher level of abstraction that makes development easier and more productive. Using RMI has therefore many advantages over using streams and sockets directly.

However, there are also some disadvantages. RMI is a synchronous protocol, while sockets and streams allow asynchronous communication. The synchronous RPC-like nature of RMI and the overhead it entails (including object serialisation and security management) make it inadequate for event-based or stream-based, data-intensive communication, e.g. as used in real-time internet video.

SAQ 4

Summarise the steps that are taken by the programmer and the runtime infrastructure to set up a remote object and then call a method on it. Take serialisation, threading and security issues into account. In particular, explain:

(a) how the registry must be set up;

(b) how the server makes the remote object available to clients and the purpose of proxies;

(c) how the client obtains a reference to the remote object;

(d) how a call to a remote object method is executed;

(e) how and why security managers are used.

ANSWER.....

(a) A registry service is started on a port known to the client and server applications. The service may run in its own JVM but, for security reasons, it must be on the same host as the server application.

(b) The server application creates a remote object and exports it, so that it is ready to receive incoming calls. The server application then asks the registry to bind (i.e. associate) a unique symbolic name to a remote reference, i.e. a reference to the remote object. The symbolic name is given by the server application and is somehow (e.g. on a public website) made known to the clients. The remote reference is a stub object, i.e. a proxy that acts as a representative of the remote object. The stub object implements the same interfaces as the remote object, but the implementation of each method just forwards the call and the arguments to the remote object.

(c) When the client wishes to make the remote call, it looks up the name in the registry. The registry returns (using serialisation) a copy of the stub object. The client now has a local reference to a stub object residing on the same JVM heap.

(d) The client makes a normal call to a method of the stub object, passing other objects or primitive data as arguments. The client thread executing the call then blocks, as it cannot continue until the call completes. The runtime support layer serialises the object arguments and uses the network support layer to send everything to the server. On the server JVM, the network and runtime support layers reverse the process, unmarshalling the arguments and making a local call to the corresponding method of the remote object. The return value or exception object is serialised and then passed back to the caller, which resumes execution.

(e) If a JVM receives a serialised object of an unknown class (i.e. a class that is not on the JVM's class path), it must use a security manager to check whether the user-defined security policy allows downloading and executing of the code from the location given in the `java.rmi.server.codebase` property set by the JVM that sent the object.

2.4 A more complex example

We will now change our example in order to take advantage of object serialisation and security managers. Instead of providing a fixed set of calculation services, such as the factorial, the computation engine will accept arbitrary tasks from the clients. This is a much more flexible and general solution that does not require the server-side code to be changed. Instead, whenever a scientist requires a new calculation service, they program the needed calculation as a task and send it to the server, so as to take advantage of its superior performance.

The task is passed from the client to the server as an object that contains all the data and methods needed to perform the task. The computation engine interface makes just one method available, with the role of executing the task it gets as argument. It returns an object containing the result. Hence the interface is simply:

```
package engine;

import java.rmi.Remote;
import java.rmi.RemoteException;

public interface ComputationEngine extends Remote
{
  public Object execute(Task t) throws RemoteException;
}
```

The server does not know in advance which tasks it will execute; all it knows is that it will receive objects that implement the following interface.

```
package engine;

public interface Task extends java.io.Serializable
{
  public Object compute();
}
```

Every object that is to be sent between JVMs has to be serialisable, and in Java this is indicated by having the object's class implementing the `Serializable` interface. That interface is actually empty (i.e. it has no methods) and serves only to signal to the JVM that object serialisation will be needed at some point in time. For our example, all tasks will be sent from the client JVM to the server JVM and therefore we declare `Task` to extend `Serializable`. In this way, any class implementing `Task` will automatically be marked for object serialisation.

Having defined the interfaces, we can start implementing the application.

The server-side code

Compared with our previous version of the computation engine, this one is simpler because it has delegated the actual task definition to the client; all there is to do is to call the task's `compute` method and return the result.

```
package engine;

import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;

public class ComputationEngineImpl extends UnicastRemoteObject
                                        implements ComputationEngine
{
  public ComputationEngineImpl() throws RemoteException
  {
    super();
  }

  public Object execute(Task t) throws RemoteException
  {
    return t.compute();
  }
}
```

Exercise 2

Why isn't the `execute` method synchronised?

Discussion .....

Each call to the computation engine passes its own serialised copy of a task object, which is therefore self-contained and independent of other task objects. Multiple concurrent calls to `execute` (and hence to `compute`) do not interfere with each other.

As for the actual server application, the only difference with respect to the previous version is that we need a security manager because the server will have to load at runtime any concrete class that implements `Task`. Such classes will be provided by the clients and therefore cannot be put in the server's class path beforehand.

We start by checking whether a security manager is already in use by the runtime system. If not, we create a new `java.rmi.RMISecurityManager` and make the runtime system aware of it.

```
package engine;

import java.rmi.RMISecurityManager;
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class Main
{
  public static void main(String[] args)
  {
    ComputationEngine engine;
    Registry registry;

    if (System.getSecurityManager() == null)
      System.setSecurityManager(new RMISecurityManager());

    // rest as in Subsection 2.2
  }
}
```

Just creating a security manager is not enough: we also have to define a security policy, but that is a topic for *Unit 10*. The activity for this subsection will include a predefined policy that grants downloaded code all possible rights, just for illustration purposes.

The client-side code

On the client side, we have to introduce a new class that calculates the factorial by implementing the `Task` interface. Because task objects will be serialised, all data to be sent to the server must be kept in instance variables and initialised by setters or constructors before the remote `execute` method is called. In the case of the factorial task, there is only one datum to initialise and store: the value of *n*.

```
package client;

import engine.Task;

public class Factorial implements Task
{
  private int n;

  public Factorial(int n)
  {
    this.n = n;
  }

  public Object compute()
  {
    long result = 1;

    for (int i = 2; i <= n; i++)
      result = result * i;
    return new Long(result);
  }
}
```

Exercise 3

Why isn't `Factorial` a remote class?

Discussion .....

For the same reason that `Task` isn't a remote interface: the tasks will be executed locally on the server. When the server calls a task's `compute` method, the task's code has been loaded into the server's JVM.

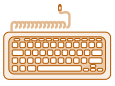
As for the `client.Main` class, only two lines of code have to change because of the new `ComputationEngine` interface and `Factorial` class. The rest remains the same.

```
package client;

import engine.ComputationEngine;
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;
import java.util.Scanner;

public class Main
{
  public static void main(String[] args)
  {
    ComputationEngine engine;
    Registry registry;
```

```
        // obtain references to registry and engine
        // as in Subsection 2.2

        try
        {
          Scanner input = new Scanner(System.in);
          System.out.print("Please input the values, ");
          System.out.println("separated by spaces, tabs or newlines.");
          while (input.hasNext())
          {
            int n = input.nextInt();
            Long result = (Long) engine.execute(new Factorial(n));
            System.out.println(n + "! = " + result.toString());
          }
        }
        catch(Exception e)
        {
          System.out.println("Factorial not computed: "
                             + e.getMessage());
        }
      }
    }
```

Several points need to be noted about this new version of the computation engine application.

- We continue to assume that the client and server are running on the same machine. In the activity we have split the client and server into two different projects so that each one has a different class path and the security manager becomes necessary.
- The client requires the `ComputationEngine` and `Task` interfaces in order to compile. Usually the developers of the server-side code make any necessary interfaces publicly available so that the client-side developers can copy them into their projects.
- On the client side, the developer has to set the `java.rmi.server.codebase` property so that when a `Factorial` object is serialised and the engine is about to call its `compute` method, the server knows where to find the corresponding class. The value of the property can be set when starting the client JVM and this has already been done for you in the activity.



Activity 7.2
Computation engine.

SAQ 5

Describe, in the correct sequence, the main steps in developing a simple client–server application using RMI. Identify in each step the interfaces or classes to be extended, the methods to be called, and, if applicable, the exceptions to be raised.

ANSWER.....

1 Write a remote interface, extending `Remote`, that declares the methods the client may call. Each method may throw a `RemoteException`.

2 Write a remote class that extends `UnicastRemoteObject` and implements the remote interface written in step 1. Remember that the methods may need to be synchronised. Furthermore, all the class constructors must eventually call the `UnicastRemoteObject` constructor, which can throw a `RemoteException`.

3 Write the rest of the server-side code. It must at some point in time (usually in `main`):
   (a) make sure that a security manager (e.g. `RMISecurityManager`) is in place if new classes have to be loaded at runtime (due to the arguments passed to remote methods);
   (b) create a registry by calling `LocateRegistry.createRegistry`;
   (c) create a remote object, i.e. an instance of the class defined in step 2;
   (d) bind the remote object to a fixed name in the registry, by calling the `bind` or `rebind` method on the registry obtained in step 3(b).
4 Write the client-side code. It must at some point in time (usually in `main`):
   (a) make sure that a security manager (e.g. `RMISecurityManager`) is in place if new classes have to be loaded at runtime (due to the return values of the remote methods);
   (b) obtain a reference to the registry by calling `Locateregistry.getRegistry`;
   (c) call `lookup` on the registry obtained in step 4(b), with the name set in step 3(d), in order to get a reference to the remote object;
   (d) call the methods on the remote object.
5 Define any necessary and appropriate security policies for the client and the server (this will be discussed in *Unit 10*).
6 Start the server JVM and one JVM per client. Set for each JVM the `java.rmi.server.codebase` property with the location of the classes of the objects that the JVM sends to other JVMs.

Note that it is possible to start the registry in a separate JVM and then use `getRegistry` in step 3(b) in the same way as in step 4(b).

3 Concurrency and distribution issues

We have looked in some detail at how RMI works, not only because it is important to be aware of the concepts and issues that have to be dealt with by a distributed communication infrastructure, but also because RMI raises some concurrency and distribution issues that have practical implications for the developers using such an infrastructure.

For example, some of the concurrency/threading issues that developers need to be aware of are as follows.

1 The thread executing the call blocks until the called remote method returns, either normally or abnormally.

2 Exporting a remote object usually creates a new thread that waits for incoming calls on that object – hence the application does not exit while such threads are running.

3 The RMI infrastructure usually creates one new thread for each incoming call; a remote method may thus be concurrently executed by multiple threads.

Note that we say 'may' and 'usually' because concrete RMI implementations are free to implement the RMI API in various ways. In particular for point 3, multiple calls to the same remote method may be executed sequentially by the same thread, but nevertheless the developer has the responsibility for making the method's code thread-safe so that it will execute correctly on any RMI implementation. As for point 2, the threads can be terminated only when the remote objects have been unexported.

As for distribution issues, three commonly recurring ones are naming, binding and locating services. In the context of RMI, services are implemented as remote methods and in the object-oriented paradigm methods have to be called through objects. Therefore in RMI the three issues amount to: unique names have to be created and made known to clients; names have to be bound to remote objects; clients have to be able to locate the remote objects by name.

The naming is defined by the server application, but the way the names are 'transmitted' to the client applications is outside RMI's scope and may have to be done by non-programmatic means, e.g. the names may be listed on the server's website for client developers to consult. Given that the client knows the remote interfaces of the services it is interested in, it is also possible for the client to `list` all the names available in the registry and then check which ones are bound to instances of the remote interfaces. The server-side code also needs to ensure that it uses unique names for its services.

The binding of names is handled by the registry, which is a remote object that holds a simple table of name–object pairs, and that, basically, provides five services: `list`, `bind`, `rebind`, `unbind` and `lookup`. The latter provides the means for clients to locate objects. It is, however, up to the client to deal with any exceptions that may arise, such as a non-existing name.

3.1 Name and directory services

Names are used by RMI to access objects located in other JVMs, but in general names are used in distributed systems to refer to a wide variety of resources, such as computers, services, databases, etc. For example, when you enter a URL into a browser

to access a website, you are requesting access to a file or a service on a remote server, by specifying only its name (the URL).

Names are more convenient and more flexible to use rather than specifying the full details of a resource or its location. It also makes it possible to change the actual location of the resource without the need to modify any components that access the resource, as long as the resource's name remains the same. All the components need is a **name** (or **naming**) **service** – a facility that allows names to be associated with resources and for resources to be accessed (looked-up) by their names. The RMI registry is an example of a name service, but you have already used many name services before, for instance every time you access a website or a file on your disk.

To access a website, we must provide its name, given by a universal resource locator (URL). A URL such as `http://www.open.ac.uk` specifies a particular web server (`www`) within a particular internet domain (`open.ac.uk`) to be accessed via a particular protocol (`http`). However, the software that controls the internet actually uses numbered IP addresses such as `194.66.152.28` to identify particular hosts. Underlying every attempt to access a host on the internet is the **Domain Name Service** (**DNS**): given a symbolic name such as `www.open.ac.uk`, it returns the corresponding numeric IP address (see Figure 3). The association between a symbolic domain name and its IP address is called a **binding** – we say the DNS binds a domain name to its IP address, just as we say that the RMI registry binds a name to a remote object. The DNS is another particular example of the general concept of a name service.

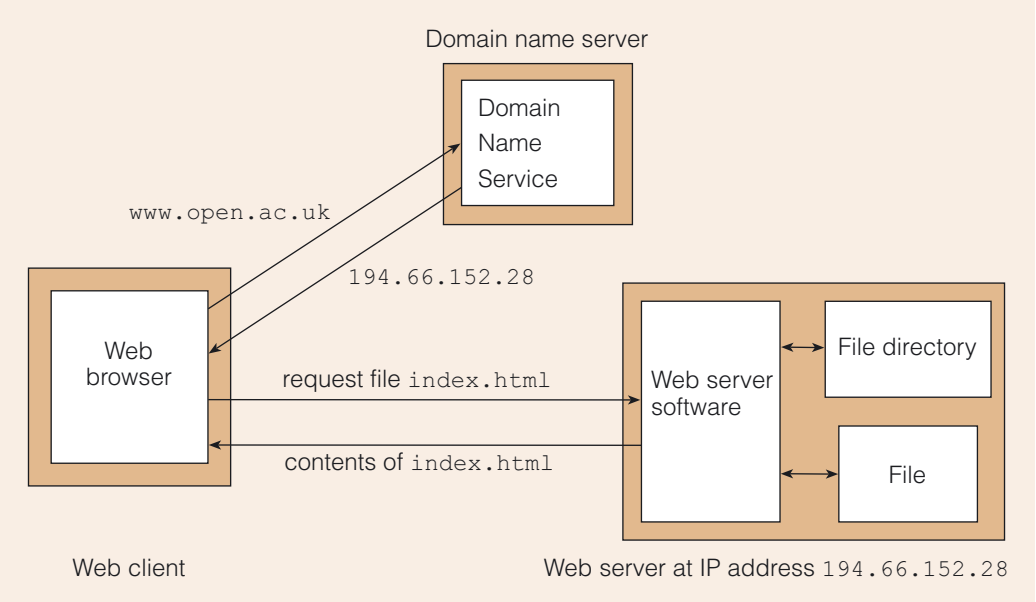


Figure 3 Web client using DNS lookup to access URL `www.open.ac.uk/index.html`

The DNS is actually provided by a whole network of computers on the internet and uses a variety of sophisticated means to ensure adequate performance. For simplicity in Figure 3 we have shown it as if it was a single computer, but actually it is a distributed system in its own right.

Another example of a name service is when we use names for the files on computer hard disks – the file system organises files into directories which bind file names to the locations of the files on disk. Files may actually be stored in a series of chunks at different physical locations on the disk – to access the file we merely specify its name and do not need to know the details of its physical location. In the example in Figure 3, the web server software looks up the location of the file named `index.html` in the file directory. All the potentially messy details of where the file is physically located are held in the file directory, and the web server software does not need to know about these details.

A file directory not only allows us to access files by names, it also provides us with information about several file attributes, like its size and the date it was last modified. A file directory is an example of a **directory service** – a facility that extends a name service with the possibility of associating various attributes to each resource and then inspecting and changing the values of those attributes. One of the advantages of directory services is that they usually allow resources to be found by attribute values (this is sometimes called 'reverse lookup'). For example, a naming service would just list an organisation's printers by name, but a directory service could include attributes like paper size and duplex capability, and then help users find all duplex A3 printers.

Terminology caveat

As often in computing, there is some vague use of language, and you may come across texts where 'name service' and 'directory service' are used synonymously, others where a directory service does not include naming facilities (i.e. only attribute manipulation) and others where a directory service subsumes a naming service. In this course we follow the last convention.

There are many vendor-specific directory services, and many of them support the internet standard Lightweight Directory Access Protocol (LDAP), a protocol to access directory services over TCP/IP.

Using unique fixed names or simple attributes is inadequate to locate and use services on a worldwide scale. Web services use instead the more sophisticated Universal Description Discovery and Integration (UDDI) standard, which goes beyond the normal capabilities of directory services. *Unit 11* will provide more details about UDDI.

3.2 Java Naming and Directory Interface

With so many different naming and directory services available, Sun Microsystems decided to include in the Java Platform a vendor-neutral API to such services. The API is called the **Java Naming and Directory Interface** (**JNDI**) and it defines a common set of functions for Java applications to access name services and directories. 'Vendor neutral' means that JNDI can be used with a wide variety of naming and directory systems, many of which have been around since before JNDI, or even before Java. The details of communicating with a particular directory system are provided by directory-specific libraries.

JNDI tutorial.

JNDI is not a service, but a set of interfaces; it allows Java applications to access many different directory service providers in a consistent way, using a standardised API. This is similar in some ways to how JDBC provides a standard interface to a variety of vendor databases. There are standard adaptors to access common naming and directory services, such as RMI, DNS, LDAP and the CORBA name service (see later in *Unit 11*).

The JNDI API is provided by the `javax.naming` package. It includes the `Context` interface which provides `bind`, `rebind`, `list` and `lookup` operations like the RMI registry. In fact, `Context` can be thought of as a generalisation of the `java.rmi.Registry` class to a wider range of naming and directory services. We will see concrete usage examples of JNDI in *Unit 9*. This section's aim has been just to show how the basic concepts and operations of RMI are an example of the wider set of name and directory services used in distributed systems for a variety of purposes.

4 Java persistence

In this and the next section we explain how the business tier for our music shop application can be programmed. As we said in the introduction to this unit, Java EE encourages the separation of data from behaviour. Data is dealt with in this section, behaviour in the next one.

Unit 6, Section 7, explained how Java programs can insert and retrieve data from a relational database using JDBC. But there is a mismatch between the 'table-oriented' way of storing data in the database and the object-oriented way of manipulating it in a Java program. The Java program must fetch data from the database using JDBC, create objects and store the data in their instance variables, call the objects' methods to manipulate the data, and use JDBC again to store any changes to the instance variables back into the appropriate tables of the database. Moreover, the Java program has to deal with transactions, performing the necessary commits. To sum up, JDBC is just a low-level mechanism for a Java program to interact with a relational database.

Component models were described in *Unit 5*, Section 4.

The goal of a framework like Java EE is to make life for developers as easy as possible. Among other things, Java EE defines a component model, called **Enterprise JavaBeans** (**EJB**), for building enterprise applications. A bean is a component written in Java, and in version 3.0 of EJB there are three types of component: session beans, message-driven beans and entity classes. Session beans encapsulate the business logic and are described in Section 5. Message-driven beans are not dealt with in this course (they are used to send messages between different enterprise applications). While session and message-driven beans provide the business logic, entity classes provide the business data – they capture the persistent business entities that the enterprise application stores in a database. For our music shop scenario, there could be entity classes to represent instruments and customers.

Entity classes are part of the generic Java Persistence API, which allows Java programs to interact easily with relational databases. The API can be used not only by the business tier but also by the web tier or even by non-enterprise applications, i.e. within the Java Standard Edition (Java SE). The API is defined by the `javax.persistence` package and provides a framework that includes three mechanisms:

- annotations to define the object/relational mapping (Subsections 4.2 and 4.3);
- an API to manipulate entities (Subsection 4.4);
- a query language to retrieve sets of entities from the database according to some search criteria (Subsection 4.5).

Moreover, the Java Persistence API promotes a style of coding whereby developers have only to provide information when sensible defaults cannot be assumed.

Entity beans

Entity classes correspond to the entity beans of version 2.1 of EJB. In the 2.1 version developers had to define multiple interfaces for each entity bean, entity beans had to extend certain classes and long configuration files were needed to define the object/relational mapping. EJB 3.0 has substantially simplified the whole process by introducing the Java Persistence API. The style of coding and the three data definition and manipulation mechanisms usually make EJB 3.0 programs much shorter and more readable than their EJB 2.1 counterparts.

The term ‘entity bean’ now refers only to pre-3.0 versions of EJB. The more general term ‘entity class’ reflects the fact that the data components can be used by themselves, i.e. without the behavioural session or message-driven beans, in various contexts besides the business tier.

The remainder of this section will briefly explain the three mechanisms listed above. The persistence framework is tightly linked to relational database concepts and technology; for example, the query language is basically a variant of SQL, of which you had a taste in the previous unit. Given that this is not a database course, we will only skim the surface of the Java Persistence API. It is a much more flexible, and hence complex, framework than our simple examples will imply. Detailed explanations and more complex examples can be found in the references given on the course website. Our main goal is just to give you an idea of what features a persistence framework, of which there are several from vendors other than Sun Microsystems, may provide to programmers in order to facilitate the development of business tiers. We start with the annotation mechanism.

EJB specification.

Java EE tutorial.

4.1 Annotations

In most programming languages, the only way to provide information about a program is in the form of comments. Comments are pieces of arbitrary text provided as documentation to human readers, and hence are marked in special ways so that compilers can ignore them. However, the designers of programming languages started to introduce new linguistic constructs that allow developers to provide information about a program that can be used by the compiler or by the runtime system. For example, such information may lead to some code being automatically generated, avoiding the need for developers to write repetitive code. The information could also be used to generate documentation in some standard way, or to make additional checks at compile time or runtime.

Version 1.5 of Java has introduced such a mechanism, called **annotations**. Various kinds of named programming construct, such as classes, methods and fields, can be annotated. Each construct may have multiple annotations. Annotations must come immediately before the programming construct they annotate, and it is good style to write them on a line of their own.

C#, a language defined by Microsoft, also has a similar mechanism.

Each annotation is an ‘instance’ of a certain type. Annotation types have a name starting with `@` and have zero or more elements, each a pair of the form *`name = value`*. Elements are given within parentheses, separated by commas. Since elements are explicitly named, they can be given in any order. The annotation type also defines which programming constructs can be annotated with it, e.g. only classes.

The following example annotates a class with an annotation of a fictitious type `@Versioning`. It has three elements: the `authors` element takes as value an array of strings, `lastChange` takes a string representing a date and `numOfChanges` an integer.

```
@Versioning(
  authors = {"John Smith", "Mary Robinson"},
  lastChange = "23/10/2006",
  numOfChanges = 6)
public class ThisClass extends ThatClass
...
```

A deprecated program element is one that programmers are discouraged from using, typically because it may lead to errors, or because a better alternative exists.

If an annotation type has no elements, the parentheses can be omitted. For example, Java predefines an `@Deprecated` annotation type without elements. This annotation type can be applied to any named programming construct and is used at runtime, e.g. to warn when a deprecated method is being called.

```
@Deprecated
public void veryOldMethod(int n)
...
```

Every annotation type is defined in a package. One must therefore use an `import` statement or write the fully qualified annotation type name. The `@Deprecated` annotation type is defined in `java.lang` and therefore you do not have to import it explicitly. Developers can also define their own annotation types, but that will not be needed in this course. We will use annotations to specify **metadata**, i.e. to provide data that describes some other data. As we will see in Subsection 4.3, metadata can be used to define the mapping from relational tables to Java classes.

4.2 Tables and classes

In a relational database, data is stored in tables, each of which usually has many rows and several columns. The **relational schema** defines the names of the tables in the database and the columns in each table. A column is defined by its name and the type of data it contains. The relational schema is an example of metadata because it defines the structure of the database, i.e. the schema is information about how the data is stored.

Figure 4 shows the data and the schema of the music shop database as used in Activity 6.1.

`ID` is a *primary key*. Each value of `ID` uniquely defines a record in *this* table.

`INSTRUMENT_MODELS` table

ID (text)	CATEGORY (text)	NAME (text)	DESCRIPTION (text)
M48-128	guitar	Bishop Edge	Bass guitar
M66-245	piano	Muzuki AG56	Upright mahogany
M84-101	trumpet	Amigo 127	Student B-flat trumpet Lacquer finish

`INST_ID` is a *primary key*. Each value of `INST_ID` uniquely defines a record in *this* table.

`MODEL_ID` is a *foreign key*. Each value of `MODEL_ID` uniquely defines a record in a *different* table, `INSTRUMENT_MODELS`.

`INSTRUMENTS` table

INST_ID (integer)	MODEL_ID (text)	PRICE (double)	SOLD (text)
1	M48-128	232.0	no
2	M48-128	182.0	no
3	M66-245	499.0	no

Figure 4 The music shop database

The ID and INST_ID columns are **primary keys**: they uniquely identify a record. The MODEL_ID column is a **foreign key**: it uniquely identifies a record in another table. In our example, each instrument is of a certain model. Instead of repeating the model's information in the instrument's record, the latter just points to the corresponding model record. This makes the database substantially smaller.

The two tables are related through the foreign key. In this case we have a **one-to-many relation** between models and instruments: for each model, there may be many instruments of that model. Inversely, instruments are in a **many-to-one relation** with models, because each instrument is of a single model and many instruments may be of the same model.

Two tables are in a **one-to-one relation** if for each record in one table there is exactly one record in the other table and vice versa. Extending our music shop database, we could impose a one-to-one relation between customers and credit cards to avoid fraudulent usage.

In a **many-to-many relation**, each record of the first table may be related to many records of the second table and, similarly, each record of the second table may be related to multiple records of the first table. In the auction scenario, each customer may bid on multiple instruments, and each instrument may have bids from multiple customers. Usually, a many-to-many relation between two tables is implemented via two one-to-many relations with a third table. For example, a many-to-many relation between customers and instruments can be defined by a one-to-many relation from a CUSTOMER table to a BID table, and another from INSTRUMENTS to BID. Each row in the BID table represents a particular bid by one customer on one instrument. The table hence includes two foreign key columns, one with the customer key and the other with the instrument key (INST_ID). A third column might represent the amount of the bid. The following table fragments (together with the INSTRUMENTS table in Figure 4) illustrate the relationship: a customer (e.g. John) may make several bids (one-to-many relation), and for each instrument (e.g. instrument 1) there may exist several bids (one-to-many relation). The net result is a many-to-many relation between customers and instruments.

Table 1 An example CUSTOMER table

CUSTOMER_ID (integer)	NAME (text)	ADDRESS (text)
1	John Smith	34 High St, Milton Keynes, UK
2	Mary Robinson	12 Main St, Smalltown, UK

Table 2 An example BID table

CUSTOMER_ID (integer)	INST_ID (text)	AMOUNT (double)
1	1	240.0
1	3	500.0
2	1	250.0

Given that many-to-many relations can be reduced to one-to-many relations, we will not consider them further in this unit.

Besides the **multiplicity** of the relationship, there is also a **direction**: relations can be **unidirectional** or **bidirectional**. The relation between models and instruments is unidirectional, because we can **navigate** it in only one direction: the instrument record points to the model, but the model does not point to the instruments of that model. In other words: the instrument 'knows' about the corresponding model, but the model does

not 'know' about the related instruments. They can be explicitly computed, by retrieving from the INSTRUMENTS table all records with the given MODEL_ID, but there is no way to access them directly from the model record.

Exercise 4

For the one-to-one example given above, involving customers and credit cards, what would each table need to include in order for the relation to be (a) unidirectional and (b) bidirectional?

Discussion

If the credit card record contains a foreign key to the customer but the customer record does not include a foreign key to the credit card, then the relation is unidirectional: it can be navigated from the card to the customer but not vice versa. The relation is also unidirectional if the customer record includes a foreign key to the credit card but the credit card does not have a foreign key to the customer. If each record has a foreign key to the other one, then the relation is bidirectional.

We now move to an object-oriented representation of the same data, which is based on classes, instance variables and objects, instead of tables, columns and records. The steps are as follows.

1 Declare one class per table.

2 Declare for each column an instance variable of the appropriate type – in the case of foreign keys the type is the class corresponding to the 'foreign table'.

3 Declare all instance variables as `private` and define appropriate getter and setter methods.

4 Define a `protected` constructor without arguments and further appropriate `public` constructors with arguments.

5 Override the `toString` method to display the primary key.

In steps 1 and 2 it is not necessary to keep the same names as in the relational schema. The note in step 2 ensures that in the same way a foreign key points to a record of another table, the corresponding instance variable refers directly to an object of the corresponding other class. The constructor without arguments in step 4 is required by the persistence framework and it must be either `public` or `protected`. We choose the latter to prevent it from being called from outside the package that defines the entities. Step 5 is recommended so that any exceptions thrown by the runtime infrastructure show a meaningful identifier, instead of the internal object identifier used by the JVM. Notice that no step above implements any kind of application logic, which should be left to session beans (Section 5).

Applying all these steps, we obtain the following class from the INSTRUMENTS table.

```
package data;

public class Instrument
{
  private int id;
  private InstrumentModel model;
  private double price;
  private String sold;

  protected Instrument()
  {
  }
```

```
  public Instrument(int id, InstrumentModel model, double price,
                    boolean sold)
  {
    this.id = id;
    this.model = model;
    this.price = price;
    setSold(sold);
  }

  public void setSold(boolean sold)
  {
    if (sold)
      this.sold = "yes";
    else
      this.sold = "no";
  }

  public boolean isSold()
  {
    return sold.equals("yes");
  }

  // further setters and getters not needed for this unit
  // instead we define just the following methods
  public String toFullString()
  {
    String msg = "Instrument " + id
                 + " (" + model.getName() + ") costs " + price;
    if (isSold())
      return msg + " (sold)";
    else
      return msg + " (in stock)";
  }

  public String toString()
  {
    return Integer.toString(id);
  }
}
```

Exercise 5

What are the differences, in terms of names and types, between the class and the table?

Discussion

Following the usual Java conventions, the class name is in the singular, while the table name is in the plural. The `INST_ID` column corresponds to the instance variable `id`, and the `MODEL_ID` foreign key corresponds to the instance variable `model` that refers directly to an instrument model. The `String` class is used to represent textual data, but the getter and setter for `sold` use Booleans instead.

Not all DBMSs support Boolean values, and in order to make our application portable, the SOLD column in the INSTRUMENTS table and the corresponding instance variable sold in class Instrument have to be textual. However, allowing users of the Instrument class to set the sold field to any string would break the application logic. The encapsulation of instance variables via getter and setter methods allows precisely the separation of conceptual state (an instrument is either sold or not) and internal representation (the strings "yes" and "no") needed in this case.

The class corresponding to the INSTRUMENT_MODELS table can be programmed in a similar way. For convenience, we add an instance variable of type Collection<Instrument> to hold all instruments of the same model. This will allow navigation of the one-to-many relation between model and instruments. Moreover, we define an enumeration to represent the possible values in the CATEGORY column. Analogous to the role of Booleans in the Instrument class, the enumeration is used by the class's interface, but the private instance variable holds a string.

Java 1.5 introduced so-called enum types to facilitate the definition of groups of constants. A typical example is to enumerate the seasons in the year: enum Season { WINTER, SPRING, SUMMER, AUTUMN }. Enum types and enumeration values can be used like classes and objects, respectively, with some predefined methods, e.g. toString.

```
package data;

import java.util.ArrayList;
import java.util.Collection;

public class InstrumentModel
{
  public static enum Category {guitar, piano, trumpet, violin};

  private String id;
  private String category;
  private String name;
  private String description;
  private Collection<Instrument> instruments;

  protected InstrumentModel()
  {
  }

  public InstrumentModel(String id, String name,
                         String description, Category category)
  {
    this.id = id;
    this.name = name;
    this.description = description;
    this.category = category.toString();
    instruments = new ArrayList<Instrument>();
  }

  public String getName()
  {
    return name;
  }

  public void addInstrument(Instrument instrument)
  {
    instruments.add(instrument);
  }
```

```
    // other getters and setters not needed; instead:
    public String toFullString()
    {
      return category + " " + id + ": " + name
            + " " + description;
    }

    public String toString()
    {
      return id;
    }
  }
```

Entities and their relationships

Classes that are meant to represent the business entities stored persistently in the relational database will be called **entity classes** and the corresponding instances, i.e. objects, will be called **entities**.

Terminology

If you consult documents from various sources about Java, including Sun Microsystems' website, you will soon notice that the use of terms varies. This is not very surprising, as it is almost impossible to force coherent terminology across many technical writers within a large organisation, let alone across organisations.

Most of the time, but not always, the official Java Persistence API specification uses 'entity' to refer to a class and 'entity instance' to refer to an object. To avoid the repetitive use of 'entity instance' we have decided to use just 'entity' instead and then make references to classes explicit.

Do not forget that in your TMAs and the examination, you are supposed to follow the course's conventions and terms, not those of other organisations or documents.

SAQ 6

Summarise how entity classes are related to the relational schema and how they are defined.

ANSWER.....

An entity class corresponds to a table, each instance of it to a record in the table and each column to an instance variable. To define an entity class for table T_1, declare one private instance variable per column and access it through a pair of setter and getter methods. If a column is a foreign key to table T_2, the corresponding variable is typed by the class corresponding to T_2. If T_1 has a one-to-many relationship to T_2, then the class may have an additional instance variable holding a collection of instances of the class corresponding to T_2.

Entity classes do not have to implement any particular interface or extend any particular class, as was the case when using RMI. However, there must be *something* in the code that distinguishes an entity class from an ordinary class, showing that entity classes are mapped to a relational schema. In the Java persistence framework, the mapping is specified through annotations, as follows.

1 Import the `javax.persistence.*` packages because they define the annotations we need.

2 Annotate the class with `@Entity` to state that it is not a 'normal' class but an entity class, i.e. the instances may be stored persistently.

3 Annotate the class with `@Table(name = string)` to provide the table corresponding to the class. This annotation comes immediately before the class declaration, after the `@Entity` annotation. If the table and class names are the same, disregarding case differences, the `@Table` annotation can be omitted.

Note the exception to the general rule of case-sensitive names in Java.

4 Annotate with `@Id` the instance variable corresponding to the primary key. Note that the variable and the key can have any name, but the annotation is always `@Id`.

5 Annotate an instance variable with `@JoinColumn(name = string)` if the corresponding column *string* is a foreign key.

6 Annotate instance variables with `@Column(name = string)` if the corresponding column name is different, except for changes in case, from the instance variable name.

We will also need to annotate the relationships between entity classes, but we will discuss how to do that later in this section. Note that the annotations are examples of metadata because they provide further information about the data contained in an entity class.

Exercise 6

Apply the steps above to writing the code for the entity classes `Instrument` and `InstrumentModel`.

Discussion

We just show where changes are made, omitting the lines that remain the same.

```
package data;

import javax.persistence.*;

@Entity
@Table(name = "INSTRUMENTS")
public class Instrument
{
  @Id @Column(name = "INST_ID" )
  private int id;
  @JoinColumn(name = "MODEL_ID")
  private InstrumentModel model;
  ...
}
```

The remaining `Instrument` instance variables do not need any annotation because they have the same case-insensitive names as the columns. For the same reason, no instance variable of `InstrumentModel` needs an `@Column` annotation. Only the primary key has to be indicated.

```
package data;

import java.util.ArrayList;
import java.util.Collection;
import javax.persistence.*;

@Entity
@Table(name = "INSTRUMENT_MODELS")
public class InstrumentModel
{
  public static enum Category {guitar, piano, trumpet, violin};

  @Id
  private String id;
  ...
}
```

It is important to realise the difference between

```
@JoinColumn(name = "MODEL_ID")
private InstrumentModel model;
```

and

```
@Column(name = "MODEL_ID")
private String model;
```

Both are correct declarations, but they mean different things. In the first case the `model` variable contains the *entity referred to by* `MODEL_ID`, while in the second case, it just contains the *textual value stored in* `MODEL_ID`.

SAQ 7

Explain whether the following declaration is correct or not.

```
@Column(name = "MODEL_ID")
private InstrumentModel model;
```

ANSWER

It is incorrect because the annotation and the type of `model` are incompatible. The `@Column` annotation implies that `model` must hold the string contained in the `MODEL_ID` column, while the `InstrumentModel` type implies that `model` must hold the record (i.e. entity) referred to by the `MODEL_ID` column. In other words, the annotation indicates that the `MODEL_ID` column should be interpreted as a normal string, while the type indicates that it should be interpreted as a key.

It should be noted that the kind of error shown in SAQ 7 cannot be checked by the Java compiler. Annotations like `@Column` are used by the Java persistence framework to map the entity classes to the relational schema. Hence any error can be detected only at runtime, when the database is actually accessed.

We now need to define how the entity classes are related. Notice that the relationships between entity classes may differ from those between the corresponding tables. For our example, the relationship between the tables is unidirectional, but the entity classes have a bidirectional relation because each class has a field (`model` and `instruments`, respectively) to obtain the corresponding instance(s) of the other class.

To describe relationships between entity classes, we annotate each field that allows us to navigate to another class. The possible annotations are `@OneToOne`, `@OneToMany` and `@ManyToOne`, and they must be chosen from the perspective of the class containing the annotation. For our example, this means that `instruments` in `InstrumentModel` is annotated with `@OneToMany` because the relation is one-to-many from that class's perspective. Inversely, `model` is annotated with `@ManyToOne`. At this point, in the `Instrument` class we have

```
@ManyToOne @JoinColumn(name = "MODEL_ID")
private InstrumentModel model;
```

and in the `InstrumentModel` class we have

```
@OneToMany
private Collection<Instrument> instruments;
```

Next we must decide which class 'owns' the relation. Basically it is the class from which we can navigate to the other *in the underlying tables*. Put in other words, the owning side is the class for which the corresponding table has a foreign key. If it is possible to navigate from either table to the other, then either of the two classes can be the **owning side**, the other being the **inverse side**. If the relation between entity classes is unidirectional, then there is obviously no inverse side, just an owning side. For our example, although the relation between the entity classes is bidirectional, the underlying table relationship is unidirectional. The owning side is the `Instrument` class because the corresponding table has the foreign key; the inverse side is the `InstrumentModel` class.

The inverse side must refer to the owning side, in particular it must say from which instance variable in the owning side (i.e. from which foreign key) it can be reached. This is done with the `mappedBy` element. It can be used with any of the multiplicity annotations given above, and its value is a string with the name of the foreign key instance variable. In our example, an `InstrumentModel` (the inverse side) can be reached via `model` (the foreign key) in `Instrument` (the owning side). At this point the annotation in `InstrumentModel` becomes:

```
@OneToMany(mappedBy = "model")
private Collection<Instrument> instruments;
```

Relationships between entities introduce dependencies. For example, it does not make sense for an instrument record to exist in the database without the corresponding instrument model record. Put differently, if an instrument model is removed from the database, all instruments of that model should also be removed, because otherwise their foreign key will point to a non-existing model. Likewise, if we create a new instrument model entity and corresponding instrument entities, if the model entity is stored to the database, the instrument entities should be stored too, automatically. The easiest way to obtain such behaviour is to use the `cascade` element to tell the runtime system which operations should be cascaded to the related entities. We want all operations (removal, storage, updates, etc.) to be propagated from a model to the corresponding instruments. The annotation in `InstrumentModel` becomes:

```
@OneToMany(mappedBy = "model", cascade=CascadeType.ALL)
private Collection<Instrument> instruments;
```

with the `CascadeType` enumeration being defined by `javax.persistence`. It is important to realise that the runtime system will not *create* the relationships automatically – it just *uses* them to store or remove related entities if the `cascade` element is set. It is therefore the developer's responsibility to explicitly relate the entities and keep the relationships consistent. For example, when a new instrument is created, it is not enough to set its `model` field; we must also update the collection of `instruments` held by the model, as we will do in the next subsection.

4.4 Managing entities

So far, the entity classes we have defined can only be used like any other Java class, i.e. we can create objects – the entities – using the constructors and then call the getters and setters. However, entities are supposed to correspond to particular rows in particular tables. How do we create an entity by retrieving a particular row from the database? How do we store any changes – made by calling the setter methods – back to the database?

The approach taken by the Java persistence framework is to use persistence contexts: a **persistence context** is a set of entities for which the runtime system knows where the data is in the persistent storage. Only when an entity is within a persistence context can any previous changes to it be made persistent, i.e. propagated to the database. Such change propagation is not automatic: it occurs only when transactions commit. Some typical scenarios will show how contexts and transactions are used.

Scenario 1 Creating a new entity and making it persistent

First, a new entity is created and made persistent:

1.1 Create a new entity by calling the constructor to initialise the entity's instance variables.

1.2 Create a persistence context.

1.3 Start a transaction.

1.4 Put the entity into the persistence context.

1.5 Commit the transaction: this will store the entity in the database.

1.6 Destroy the persistence context if it is no longer needed.

Note that destroying a persistence context does not destroy the entities within it; it just severs the mapping between the entities and the persistent storage. The entities continue to exist as Java objects, but the runtime system no longer knows to which rows of which tables they correspond.

Scenario 2 Changing existing entities and making changes persistent

We can change the entities and store the changes back to the database in basically the same way as above:

2.1 Call the setters on an existing entity to change its state.

2.2 Create a persistence context if no context currently exists.

2.3 Start a transaction.

2.4 Put the entity into the persistence context, if it was not already there.

2.5 Commit the transaction: this will update the corresponding rows in the database.

2.6 Destroy the persistence context if it is no longer needed.

Note the remark in step 2.4: once an entity is put in a persistence context, it stays there until the context ceases to exist.

Scenario 3 Deleting data from the database

Another typical scenario is to delete some data from the database:

3.1 Create a persistence context if no context currently exists.

3.2 Make a query on the database: this will return a set of entities that are automatically within the currently existing context.

3.3 Start a transaction.

3.4 Mark each of the retrieved entities for removal.

3.5 Commit the transaction: this will delete the corresponding records in the database.

3.6 Destroy the persistence context if it is no longer needed.

In the next subsection we will see how to program step 3.2 with the Java Persistence API; in this subsection we cover only the other steps. Moreover, we assume that we are using the framework within a Java SE context – we will develop our first Java EE application in Section 5.

The lifecycle of an entity

To perform the above steps we need **entity managers**: through them, an application can manage an entity's lifecycle. In practical terms, the `javax.persistence.EntityManager` interface provides methods to put entities into persistence contexts and, among other things, mark them for removal. The lifecycle of an entity has four states: New, Managed, Detached and Removed. If an entity is in the New or Detached state, it is outside a persistence context. Vice versa, if an entity belongs to a persistence context, it must be in the Managed or Removed state. Figure 5 shows the states and which entity manager methods trigger which state transitions.

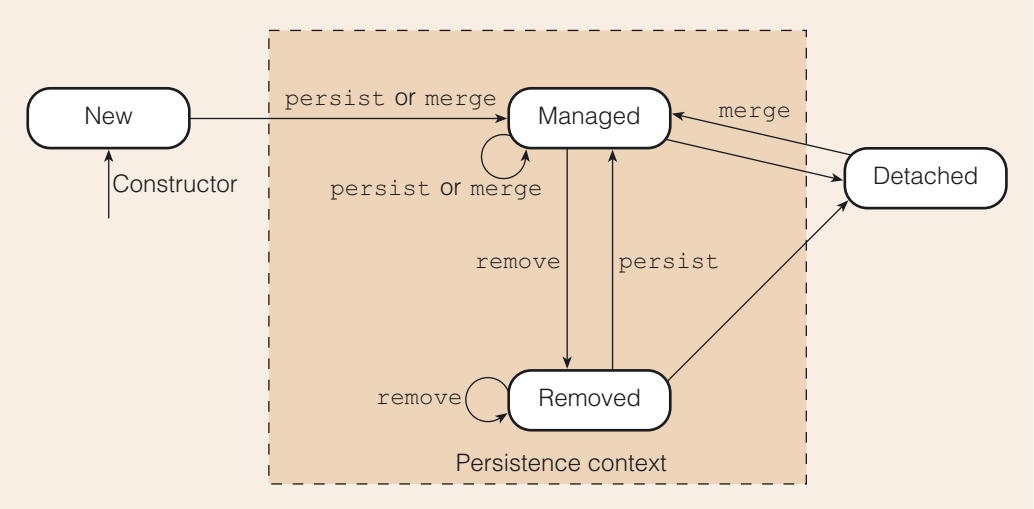


Figure 5 The lifecycle of an entity within and outside a persistence context

When an entity is created, by calling a constructor, it starts in the New state. An entity manager's `persist` method puts a new entity into the persistence context. The entity's state becomes Managed. When a transaction commits, managed entities are stored in the database. Calling the `remove` method on a managed entity puts it into the Removed state. Removed entities are deleted from the database when a transaction commits. The `persist` and `remove` calls are always propagated to related entities if `cascade=CascadeType.ALL` is set. Calling `persist` on a removed entity puts it back into the Managed state. Calling `persist` on a managed entity or calling `remove` on a removed entity has no effect on the entity, apart from propagation to any related entities. Managed entities become detached when the persistence context ceases to exist, and we will say more about how this happens later. The application can call the methods (e.g. setters) of a detached entity, but any changes to it are not reflected in the database. Calling `remove` or `persist` on a detached entity raises an exception, but the `merge` method puts a detached entity back into the Managed state. On new and managed entities, `merge` has the same effect as `persist`.

The concept of a detached entity is important because it reflects one of the issues in distributed systems, namely that the state of various parts of the system may not be consistent. We have already come across this problem in *Unit 6*, Section 8, when we discussed distributed databases. Having a class-based representation of the data in

the application and a table-based representation in the database is conceptually not very different from having two table-based representations. The important fact is that data is replicated, and changes made in one place should be reflected in the other. This in turn relates to our discussion of serialisation in Subsection 2.3. A detached entity is like a serialised object that is copied from one JVM to another: any changes to it are not reflected in the 'original' object. In this case, changes to a detached entity are not reflected in the database record – unless the entity is merged again into a persistence context.

Returning to Figure 5, as the two unlabelled arrows into the Detached state show, entities become detached implicitly – i.e. without any explicit call to an entity manager method – when the persistence context ceases to exist. In Java SE applications, the class of applications to which this discussion applies, persistence contexts are automatically associated with entity managers. Creating a manager creates a context, and 'closing' a manager destroys the associated context. Once a manager is closed, via the `EntityManager.close` method, no further methods can be called on it.

Besides managing entities and providing persistence contexts, entity managers also provide an interface to the underlying transaction mechanism. The `EntityManager` interface declares a method `getTransaction` which returns an `EntityTransaction` object. Such objects have methods `begin`, `commit` and `rollback` with the obvious meanings.

We can now provide the Java code needed for the scenarios discussed at the start of this subsection. Using equivalent numbering to that used earlier, the creation and storage of new entities (scenario 1) becomes:

1.1 Create a new entity `E`, e.g. `Instrument E = new Instrument(...)`.

1.2 Create an entity manager `M`: this creates a persistence context.

1.3 Call `M.getTransaction().begin()` to start a transaction.

1.4 Call `M.persist(E)` to put the entity into the persistence context.

1.5 Call `M.getTransaction().commit()` to store the entity in the database.

1.6 Call `M.close()` to destroy the persistence context.

The other scenarios are similar, the major difference being the calls to `merge` in step 2.4, and to `remove` in step 3.4, instead of calling `persist` as in step 1.4.

Creating entity managers

All that remains is to see how to create entity managers (steps 1.2, 2.2 and 3.1). First we have to define the **persistence unit**, which specifies the database to be accessed and the entity classes that define the mapping between objects and database records. Persistence units are defined with XML files, the details of which are irrelevant for our purposes. Each persistence unit has to declare its name. Given that name, the following Java code creates an entity manager.

```
import javax.persistence.*;
...
EntityManagerFactory factory =
  Persistence.createEntityManagerFactory(/*persistenceUnitName*/);
EntityManager manager = factory.createEntityManager();
...
manager.close();
factory.close();
```

As you can see, first we create an entity manager factory associated with the given persistence unit and then we call a factory method to obtain a new entity manager. Like the entity managers, the factory has to be closed when it will not be further used.

Factory methods were introduced in *Unit 4*, Subsection 3.6.

This will signal to the Java persistence runtime that resources (e.g. database connections) can be released.

An example: creating and storing new entities

We are now in the position to show a complete Java SE application that uses the Java Persistence API. The example creates an instrument model and one instrument, according to the records shown in Subsection 4.2, and stores them in the database. We assume the persistence unit is called `music`.

```
package data;

import data.InstrumentModel.Category;
import javax.persistence.*;

public class Main
{
  private static void addStradivarius(EntityManager manager)
  {
    InstrumentModel model = new InstrumentModel(
            "Strad", "Stradivarius",
            "Professional high-quality violin",
            Category.violin);
    Instrument instrument = new Instrument(101, model, 3000, false);
    model.addInstrument(instrument);

    EntityTransaction transaction = manager.getTransaction();
    transaction.begin();
    manager.persist(model);
    transaction.commit();
  }

  public static void main(String[] args)
  {
    EntityManagerFactory factory =
            Persistence.createEntityManagerFactory("music");
    EntityManager manager = factory.createEntityManager();

    addStradivarius(manager);
    manager.close();
    factory.close();
  }
}
```

Note that we set both sides of the model–instrument relationship, i.e. we not only state the model of the instrument but also add the instrument to the collection held by the model. We declare an explicit transaction object to avoid getting it repeatedly from the entity manager.

Exercise 7

Why is there only one `persist` call although two entities (an instrument and a model) were created?

Discussion .....

There is only one call because the `persist` operation on the model is automatically propagated to the related instrument. Recall that in Subsection 4.3 we defined the `InstrumentModel` entity class in such a way that all operations should be cascaded along the one-to-many relationship:

```
@OneToMany(mappedBy="model", cascade=CascadeType.ALL)
private Collection<Instrument> instruments;
```

4.5 Finding entities

Once entities are stored in the database they can be retrieved. An entity manager provides a method `find` to obtain an entity by primary key. The method returns `null` if no entity with the given key exists. The method has two arguments: the first is the entity class, i.e. the type of object to be found, and the second is the primary key value. Note that classes cannot be passed as arguments of methods, only objects. Java and other languages circumvent this by having special objects that represent classes. There is exactly one such special object for each class. In Java, each class has a static variable, appropriately named `class`, which refers to that special object.

As an example we change the previous program to first check if the database already has the Stradivarius model before inserting it. Otherwise we will get a runtime error when trying to insert a new entity with the same primary key. The code in the `main` method becomes:

```
if (manager.find(InstrumentModel.class, "Strad") == null)
        addStradivarius(manager);
```

However, the primary key is often some obscure code that users should not be required to remember. Normally, users find entities by their name, description, etc. The SQL language allows expressive queries to be run on relational databases. The Java Persistence API includes a query language that closely mimics SQL, but uses entity classes and their instance variables instead of table and column names.

Entity managers provide a factory method `createQuery` that takes as argument a string with the query and returns a `Query` object, which then accepts a call to `getResultList` in order to execute the query and return a list of the found entities. An example is the best way to see how it works. The following method can be added to the `Main` class of Subsection 4.4 in order to select all models currently in the database and then print each one out, by iterating over the resulting list.

```
public static void printModels(EntityManager manager)
{
  Query query =
        manager.createQuery("SELECT m FROM InstrumentModel m");
  List<InstrumentModel> models =
                  (List<InstrumentModel>) query.getResultList();

  System.out.println("Instrument models:");
  for (InstrumentModel model : models)
    System.out.println(model.toFullString());
}
```

Note that `getResultList` has a declared return value of `List<Object>` and therefore the cast is needed.

A more interesting example is to select only entities that match some user-defined characteristics. The following method displays all instruments that belong to a given category and are below a given price. This allows the user to ask, for example, for all guitars up to 200 monetary units.

```
private static void findCheapInstruments(EntityManager manager,
        String category, double maxPrice)
{
  System.out.println(category + " instruments up to "
                     + maxPrice + ":");
  Query query = manager.createQuery(
          "SELECT i FROM Instrument i WHERE i.model.category = '"
          + category + "' and i.price <= " + maxPrice);
  for (Instrument i : (List<Instrument>) query.getResultList())
    System.out.println(i.toFullString());
  System.out.println("no further results");
}
```

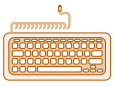
Given that a query is just a string, we can build it dynamically at runtime with Java's concatenation facilities, in order to customise it with user-defined parameters. If a query parameter is a string, it has to be enclosed in single quotes, as done above.

Notice that the `select` statement we have seen in *Unit 6*, Section 7, has a slightly different syntax because we are now using entity classes and their instance variables, not tables and columns. Although the instance variables are private, we can use them to navigate through the entities, with the usual Java syntax. For example, given an instrument `i`, the expression `i.model.category` accesses the `String` object in the `category` field of the `model` entity associated with the instrument `i`, and we can thus compare it to the user-provided string.

Querying a database does not change it. Queries can therefore be executed outside a transaction, to make the application more efficient. The entities returned by the query are in the Managed state because there is a current persistence context (associated to the entity manager used to do the query). Therefore, if after the query we start a transaction, change the retrieved entities and commit the transaction, the changes will be made persistent in the database without any need for an explicit call to `merge` or `persist` because the entities are already managed. This is basically the same as the third scenario at the start of Subsection 4.4, but updating entities instead of removing them. The following code template illustrates the retrieval and update/removal of entities.

```
void changeSomeEntities(EntityManager manager)
{
  Query query = manager.createQuery(...);
  List results = query.getResultList();

  manager.getTransaction().begin();
  for (Object entity : results)
  {
    // update the entity by calling some setter methods
    // or remove it by calling manager.remove(entity)
  }
  manager.getTransaction().commit();
}
```

The activity for Section 4 contains the complete code of the `Main` class, including the above query methods, and it asks you to add an update method following the above template.



Activity 7.3
Music Shop entities.

The entity managers used in this section are said to be **application managed** because they are explicitly managed – i.e. created and closed – by the application. The persistence contexts we used are said to be **extended** because their existence is associated with the entity manager and as such can span multiple transactions.

In this section we wanted to show that it is possible to use the Java Persistence API in Java SE applications. In the next section we will use other kinds of entity manager and persistence context that are more suitable for Java EE applications.

For your reference, we conclude with a summary of the `javax.persistence` classes, methods and annotations that we have used in this section.

Table 3 `javax.persistence` annotations and enumerations

Annotation/enumeration	Description
`@Entity`	The annotated class is an entity class.
`@Table(name = string)`	The annotated class corresponds to the table *string*.
`@Id`	The annotated instance variable corresponds to the primary key.
`@JoinColumn(name = string)`	The annotated instance variable holds the entity referred to by the foreign key *string*.
`@Column(name = string)`	The annotated instance variable corresponds to the column *string*.
`@OneToOne(mappedBy = string, cascade = operations)`	The annotated instance variable refers to a single foreign entity, which in turn refers, via its instance variable *string*, back to this entity. The `mappedBy` attribute is specified only if the foreign entity owns the relation. The given *operations* are propagated from this entity to the foreign entity.
`@OneToMany(mappedBy = string, cascade = operations)`	The annotated instance variable refers to multiple foreign entities, which in turn all refer, via their instance variable *string*, back to this entity. The `mappedBy` attribute is specified only if the foreign entities own the relation. The given *operations* are propagated from this entity to the foreign entities.
`@ManyToOne(mappedBy = string, cascade = operations)`	The annotated instance variable refers to a single foreign entity, which in turn refers, via its instance variable *string*, back to this entity and possibly other instances of this entity class. The `mappedBy` attribute is specified only if the foreign entity owns the relation. The given *operations* are propagated from this entity to the foreign entity.
`CascadeType`	An enumeration with values `ALL`, `PERSIST`, `MERGE` and `REMOVE`, to indicate which operations should be propagated.

Table 4 `javax.persistence.EntityManager` interface

Method	Description
`void close()`	Closes an application-managed entity manager.
`Query createQuery(String aQuery)`	Constructs a query built from the given string.
`EntityClass` `find(Class` *`EntityClass`*`, Object keyValue)`	Returns the unique entity, of the given class, that has the given primary key value. If no such entity exists, the method returns `null`.
`EntityTransaction getTransaction()`	Returns a transaction.
`EntityClass` `merge(`*`EntityClass`* `entity)`	Merges the state of the given entity into the current persistence context, returning a managed copy of the entity.
`void persist(Object entity)`	Makes the given entity managed and persistent.
`void remove(Object entity)`	Removes the given entity.

Table 5 Other methods

Interface or Class	Method	Description
`EntityManagerFactory`	`EntityManager createEntityManager()`	Returns an application-managed entity manager.
`EntityManagerFactory`	`void close()`	Closes the factory of entity managers.
`EntityTransaction`	`void begin()`	Starts a transaction.
`EntityTransaction`	`void commit()`	Commits the current transaction, making any changes to the database.
`EntityTransaction`	`void rollback()`	Rolls back the current transaction.
`Persistence`	`static EntityManagerFactory createEntityManagerFactory(String unitName)`	Returns a factory of entity managers for the given persistence unit.
`Query`	`List getResultList()`	Executes the query, which must be a `select` statement, and returns the list of found entities.

5 Session beans

This section looks at session beans: they are objects that contain the application logic, providing business services to clients. In this unit we are assuming that the clients are independent applications – other kinds of client are considered in *Units 8* and *9*.

Before we move on to technical details, some explanation about terms used henceforth is in order. As is the case with the term 'entity', it is possible to find documents where 'bean' means an object and others where it refers to a class. We will use the former convention because it saves us from constantly repeating the more cumbersome term 'bean instance'. We will also follow the widely adopted convention of ending bean class names in `Bean`, e.g. `ShoppingCartBean`. This is just the application of the more general convention of naming classes after what their instances represent (e.g. `Person` instead of `PersonClass`) to the case where an instance is called 'bean'. Moreover, in *this section only*, 'bean' will always refer to a session bean.

5.1 Introduction

Session beans access and modify entities, shielding data from clients. Session beans are one kind of Enterprise JavaBean (EJB). All the classes, interfaces and annotation types relating to session beans are provided by the `javax.ejb` package. Session beans run inside an **EJB container**. The container is a runtime environment, provided by the Java EE application server, that supports system-level services such as transaction and security management. This makes it easier for developers to concentrate just on the application's business logic.

A session bean, as the name suggests, encapsulates an interactive session of a single client with the application server. An interaction consists of the client calling one of the bean's methods and possibly getting a result back. A session is therefore a 'conversation' (i.e. a sequence of one or more method calls and returns) between client and session bean. Multiple clients may be accessing the same data, i.e. entities are persistent and shared, but each client will do so through a distinct session bean – session beans are not shared among clients. Moreover, once the session terminates, the bean is no longer associated with the client and can be discarded – session beans are not persistent. There are two kinds of bean, **stateful** and **stateless**.

In a stateful bean, the state (i.e. the value of the bean's instance variables) is kept across multiple calls to the bean's methods. A classic example is a shopping cart in an online shop. The shopping cart has to keep its state (i.e. the products it contains) across multiple interactions with the client, who might add and remove products from the cart.

In a stateless bean, the state is discarded after each method call, i.e. a session with a stateless bean lasts only for the duration of a single method call. This makes stateless beans ideal to provide one-off services that are performed always in the same way, independently of the actual client invoking it. For example, a database query can be seen as a complete session by itself, and it is not necessary to keep a record of which client executed which query. Therefore a single stateless bean can provide multiple methods, each one implementing an independent query.

Another way of saying that stateless beans forget their state after each method call is to say that all instances of the same stateless bean class are equivalent except during method calls. This means that the EJB container may wish to keep a pool of stateless

beans and, whenever a client calls a method, assign any available bean to that client in order to execute the method. Once the method terminates, the bean can be returned to the pool.

SAQ 8

What is the rationale for using a pool of stateless beans? You may wish to remind yourself of the thread pools discussed in *Unit 4*, Subsection 3.6.

ANSWER.....

The rationale for using pools of resources is always the same: performance. Requests can be serviced faster by an already available and reusable resource, instead of creating a new one when the request arrives and then garbage collecting it after the request terminates.

A session bean goes through various stages during its lifetime. A stateless bean is either non-existent or ready. A bean is ready when it can accept method calls. It becomes ready after being created and becomes non-existent when it is removed. A stateful bean has a third stage: passive. It moves from the ready to the passive stage when it is written to disk, and moves back to the ready stage when it is read from disk.

The reason for this is that a stateful bean lasts for as long as the client is interacting with the server, which means that there may be a long delay between method calls. For example, a user may be browsing various instruments, checking carefully each one's description, before deciding on which one to add to the shopping cart. This means there will be several calls to stateless beans to query the music shop's database between any two consecutive calls to the shopping cart stateful bean. To make efficient use of memory, when many clients are interacting with the server the EJB container may decide to write a stateful bean to disk after a method call, and retrieve it only when the next call is made. Serialising objects to disk and reading them back into memory entails a performance penalty. Stateless beans, on the other hand, are never written to disk because no conversational state has to be kept: if memory becomes scarce, the container may, for example, just reduce the pool size (i.e. let the JVM garbage collect some stateless beans), and then create new instances only when they are needed.

5.2 Business interfaces

Similarly to remote objects (Subsection 2.2), beans are defined by a **business interface** that describes the client's view of the bean, and by a class that implements the interface and any additional helper methods. There may be, of course, further helper classes, e.g. to define any application-specific exceptions raised by the bean's methods. In general, it is possible for one bean class to implement various business interfaces, and for a business interface to be implemented by different bean classes, but keeping a one-to-one correspondence between a business interface and a bean class avoids subtle problems that may occur.

The business interface not only declares the methods available to clients, it also states whether clients will access the bean remotely or locally.

SAQ 9

What do the terms remote access and local access mean?

ANSWER.....

If access is remote, client and bean are executing on different JVMs; if access is local, they are executing within the same JVM.

Deciding on whether to allow remote or local access depends on various factors:

- *Client type*. Bean methods can be called by stand-alone client applications, by web tier components (explained in the next unit) or by other beans. If the client is an independent application, then access must be remote, because separate client and server applications run on separate JVMs. If the client is another bean or a web component, the access mode can be either local or remote.
- *Distribution*. If client and bean may potentially run on different machines, remote access must be provided.
- *Performance*. Remote access is slower than local access.
- *Data isolation*. If client and bean data need to be isolated from each other, remote access should be chosen, to enforce the serialisation of method parameters and return values. This will prevent any accidental change to parameters by the bean method being reflected in the client, and vice versa for the returned object.
- *Data granularity*. Given that remote calls are slower than local calls, data passed between client and bean should be more coarse grained so that fewer calls are needed for a complete session.

Note that distribution and performance may be related. For example, if the client of a bean is a web tier component, putting the web tier and the business tier on different machines may improve overall responsiveness to the user, even if the method calls from components in the web tier to the beans in the business tier take longer than having both tiers on the same machine. Only a system-wide performance test under actual usage conditions can provide insight into how best to distribute components among hosts.

Business interfaces are defined like any other Java interface. The major difference is that they have to be annotated with `@Remote` or `@Local` to indicate the kind of access. Moreover, methods must be `public` – they cannot be `static` or `final` – and their names cannot start with `ejb` to avoid clashes with predefined methods.

SAQ 10

In this unit we assume that the client is a separate application – web clients are covered in the next unit. How will session beans be annotated?

ANSWER.....

They will be annotated with `@Remote`, because separate client applications do not run on the same JVM as the server, as stated under the ‘Client type’ bullet item above.

As an example, the following business interface declares queries to obtain, as a formatted string, the models and instruments currently in the database.

```
package session;

import javax.ejb.Remote;

@Remote
public interface QueriesRemote
{
  public String getModels();
  public String getInstruments();
}
```

Notice that the methods do not throw a `RemoteException`: the container will throw an unchecked `EJBException` (or subtype thereof) if something goes wrong during the remote call.

5.3 Stateless session beans

Stateless bean classes are defined like any other Java class, but they have to be annotated with `@Stateless`. Access to the database can be done as shown in Subsections 4.4 and 4.5:

1 the bean's constructor creates an entity manager;

2 each bean's method calls the necessary entity manager methods to retrieve, change and store entities;

3 the entity manager is closed when the bean is no longer necessary.

In Section 4 we used application-managed entity managers because Java SE does not provide the same runtime support infrastructure as Java EE, but in this section we will delegate the creation and disposal of entity managers to the EJB container, i.e. we will use **container-managed entity managers**. All we have to do is to indicate that the bean requires an entity manager and the container will create a new one or reuse an existing one. This is called **dependency injection**: an object (the bean in this case) indicates that it depends on the existence of another object (an entity manager in this case) and the container 'injects' such an object into the application at runtime. Dependency injection is usually achieved by annotating the declaration of the required object. The container then uses the information provided by the annotation to create a new object (or reuse an existing one). In this case, we declare an entity manager and provide only the persistence unit, via an annotation. We do not call the manager's constructor because the object will be injected by the container. The following implementation of the `QueriesRemote` business interface shows what the developer has to do.

```
package session;

import data.Instrument;
import data.InstrumentModel;
import java.util.List;
import javax.ejb.Stateless;
import javax.persistence.*;

@Stateless
public class QueriesBean implements QueriesRemote
{
  @PersistenceContext(unitName="music")
  private EntityManager manager;

  public String getModels()
  {
    StringBuffer result = new StringBuffer("Instrument models:\n");

    Query query =
          manager.createQuery("SELECT m FROM InstrumentModel m");
    List<InstrumentModel> models =
                    (List<InstrumentModel>) query.getResultList();

    for (InstrumentModel model : models)
      result.append(model.toFullString() + '\n');
    return result.toString();
  }

  public String getInstruments()
  {
   // similar to getModels
  }
}
```

Note that there is no explicit initialisation and closing of the `manager` variable, nor is there any entity manager factory that has to be created and closed: the EJB container will take care of all that.

Within the Java persistence framework, if the EJB container manages the entity managers, it will usually also handle the transactions. This means that the developer usually does not state when transactions begin and commit within a method. Instead, *all* of the code within a method is supposed to execute either within a single transaction or within no transaction at all. By default, if a transaction is active when a bean method is called, the method is executed within that transaction. Otherwise, a new transaction is started, and it commits just before the method returns.

It is possible to change this default behaviour by annotating the bean methods or the bean class with `@TransactionAttribute`, which takes as argument one of the `TransactionAttributeType` enumeration values. Annotating the class changes the behaviour of all its methods; annotating a method changes only that method's behaviour. There are several possible behaviours, but the two most useful ones are given by the following `TransactionAttributeType` values.

- `REQUIRED`. This is the default behaviour: a transaction is required by this method, and either the currently active one is used or a new one is started.
- `NOT_SUPPORTED`. The method will not be executed within a transaction. Hence, if one is active, it is suspended for the execution of the method.

To illustrate this, we will state that by default query methods do not require transactions, because they only read the database. The changes to the bean class are:

```
package session;
...
import javax.ejb.TransactionAttribute;
import javax.ejb.TransactionAttributeType;

@Stateless
@TransactionAttribute(TransactionAttributeType.NOT_SUPPORTED)
public class QueriesBean implements QueriesRemote
{
   ...
}
```

Changing from application-managed to container-managed entity managers changes not only how transactions are handled but also the lifetime of persistent contexts. By default, they exist for only the duration of transactions, i.e. the persistence context is created when the transaction begins, and ceases when the transaction commits or rolls back. This means that if transactions are `NOT_SUPPORTED` by a bean method, any entities manipulated within that method are detached because without a transaction there is no persistence context. However, stateless bean methods usually do not change entities – due to the nature of stateless beans – and hence it is not a problem for entities to be detached in such methods, as in the example above.

If instead of the default **transaction-scoped persistence contexts** we want to use extended persistence contexts with container-managed entity managers, then the annotation would be as follows.

```
import javax.persistence.PersistenceContextType;
...
@PersistenceContext(unitName="music",
                    type=PersistenceContextType.EXTENDED)
private EntityManager manager;
```

SAQ 11

(a) What are the two kinds of entity manager?

(b) What are the two kinds of persistence context?

(c) Which kinds of persistence context can be used with which kinds of entity manager?

ANSWER.....

(a) The two kinds of entity manager are:

- application-managed entity managers are explicitly created and closed by the application, and they explicitly start and end transactions;
- container-managed entity managers are implicitly created and closed by the EJB container, and transactions are by default implicitly created and committed at method call and return.

(b) There are two kinds of persistence context:

- extended contexts are created when an entity manager is created (by the application or the container), and cease to exist when the entity manager is closed (by the application or by the container);
- transaction-scoped contexts are created when a transaction starts, and cease when the transaction commits or rolls back.

(c) Persistence contexts are always extended if the entity managers are application managed, whereas both kinds of context can be used with container-managed entity managers.

SAQ 12

At the start of Section 4, we said that using the Java Persistence API promotes a succinct coding style that favours using defaults. Name some of those defaults.

ANSWER

By default, the Java Persistence API assumes that:

1 a table has the same name as the entity class, and the columns have the same names as the instance variables;

2 persistence contexts are transaction scoped when using container-managed entity managers;

3 when using container-managed entity managers, each method executes within the current transaction, or else starts a new transaction and commits it when the method terminates.

Coding becomes simpler because fewer annotations are required (due to assumption 1) and calls like `manager.getTransaction().begin()` are unnecessary (due to assumption 3). Moreover, assumptions 2 and 3 together mean that within a method all entities are either New/Detached or Managed/Removed. This makes calls to `merge` or `persist` necessary in only a few cases, e.g. if entities are passed from a method that does not execute within a transaction (`NOT_SUPPORTED`) to one that does (`REQUIRED`).

5.4 Stateful session beans

Stateful beans can retain information across a 'conversation', i.e. multiple method calls. The state of stateful beans is therefore sometimes called the *conversational state*. In our online application for The Music Store, we will use a stateful bean to represent a shopping cart, because each interaction can add or delete one instrument to/from the cart and we have to keep track of the previous interactions, i.e. which instruments are in the cart. A stateful bean class is defined like a stateless one, but annotated instead with `@Stateful`.

The following code shows the business interface of shopping cart beans. We only declare a method to add an instrument, but methods to remove one or all instruments or to view the contents of the cart should also be provided. The `addInstrument` method returns `false` if the instrument, given via its primary key value, could not be added to the cart for some reason. The last method of the session confirms that we indeed wish to order the instruments in the cart.

```
package session;

import javax.ejb.Remote;

@Remote
public interface CartRemote
{
  public boolean addInstrument(int id);
  public void confirmOrder();
}
```

Typically, a stateful bean provides one or more constructors to initialise the session (to be provided in the implementation of `CartRemote`), one or more methods to use during the conversation (`addInstrument`), and one method to signal the end of the session (`confirmOrder`), which has to be annotated with `@Remove` so that the EJB container knows that the bean can be removed (i.e. garbage collected) after that method terminates, either normally or abnormally. Any further call to that bean's methods will raise an exception.

We will implement the cart as a list of integers, to represent the primary keys of the instruments in the cart. An instrument can be added to the cart provided that it has not yet been sold. However, until the customer confirms the purchase, its state remains as unsold, so that a number of clients shopping simultaneously may add it to their carts. Hence, when a client confirms the order, the current instrument status is checked again, to see if it has been sold meanwhile to another customer, and a message is printed accordingly.

```
package session;

import data.Instrument;
import java.util.ArrayList;
import javax.ejb.Remove;
import javax.ejb.Stateful;
import javax.persistence.EntityManager;
import javax.persistence.PersistenceContext;

@Stateful
public class CartBean implements CartRemote
{
  @PersistenceContext(unitName = "music")
  private EntityManager manager;

  private ArrayList<Integer> items;

  public CartBean()
  {
    items = new ArrayList<Integer>();
  }

  public boolean addInstrument(int id)
  {
    Instrument i = manager.find(Instrument.class, id);
    if (i == null || i.isSold())
      return false;
    else
    {
      items.add(id);
      return true;
    }
  }

  @Remove
  public void confirmOrder()
  {
    for (int id : items)
    {
      Instrument anInstrument = manager.find(Instrument.class, id);
```

```
            if (anInstrument.isSold())
              System.out.println(id + " was sold to another customer");
            else
            {
              anInstrument.setSold(true);
              System.out.println(id + " sale confirmed");
            }
          }
        }
      }
```

Exercise 8

Explain how the code above works, in particular the use of annotations and transactions. Would you change the default transaction setting?

Discussion .....

The `@Stateful` annotation notifies the compiler that the otherwise 'normal' `CartBean` class implements a stateful bean. The constructor initialises the cart's `items` to the empty list.

Each call to `addInstrument` has to first check if the given primary key exists and if the corresponding instrument has not been sold yet. If both conditions are met, the primary key is added to the list and the method returns `true`, otherwise it returns `false` without changing the state of the cart. The method uses the `find` method to retrieve an entity by its primary key. This requires an entity manager, which is created by dependency injection through the `@PersistenceContext` annotation.

The `confirmOrder` method is annotated with `@Remove` so that the EJB container knows that the session is over once this method terminates. The method simply traverses the list with a *`for-each`* loop and checks whether the instruments in the cart have been sold meanwhile, again using the entity manager's `find` method. Each unsold instrument becomes sold, and appropriate messages are printed.

By default, methods `addInstrument` and `confirmOrder` execute within an existing or newly created transaction, as if the class had been annotated with `@TransactionAttribute(TransactionAttributeType.REQUIRED)`. The `addInstrument` method does not change the database and hence could be executed outside a transaction with the following annotation.

```
@TransactionAttribute(TransactionAttributeType.NOT_SUPPORTED)
public boolean addInstrument(int id)
```

However, `confirmOrder` does change the database and needs to be executed within a transaction. This is to ensure that if an instrument belongs to multiple orders which are confirmed simultaneously, it is sold only once.

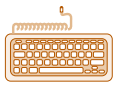
5.5 Using session beans

Session beans can be used in application clients in a straightforward way: the client obtains a reference to the bean by dependency injection via the `@EJB` annotation and then calls its methods. Obviously, the client has access to only the bean's business interface and the methods declared therein. The following example shows the contents of the database and then accepts a sequence of instruments to put in the cart,

by reading integers from the standard input. When the input finishes, the order is confirmed and the new state of the database is shown.

```
package client;

import java.util.Scanner;
import javax.ejb.EJB;
import session.CartRemote;
import session.QueriesRemote;

public class Main
{
  @EJB
  private static QueriesRemote queryBean;
  @EJB
  private static CartRemote cartBean;

  public static void main(String[] args)
  {
    System.out.println(queryBean.getModels());
    System.out.println(queryBean.getInstruments());

    Scanner input = new Scanner(System.in);
    System.out.println("Please indicate instruments"
                       + " to add to the cart.");
    System.out.println("Separate instrument ids with"
                       + " spaces or newline.");
    System.out.println("Close input to terminate shopping.");
    while (input.hasNext())
    {
      int id = input.nextInt();
      if (!cartBean.addInstrument(id))
        System.out.println("Instrument unavailable: " + id);
      else
        System.out.println("Instrument added to cart: " + id);
    }
    cartBean.confirmOrder();

    System.out.println(queryBean.getInstruments());
  }
}
```

Note that while entity managers are injected with the `@PersistenceContext` annotation, Enterprise JavaBeans are injected with the `@EJB` annotation. You are now in a position to do the activity, which will ask you to study the full code of our example, guide you through the steps to build and run your first Java EE application, and then ask you to modify the code to introduce a business rule to provide price discounts.



Activity 7.4
Session beans.

We can sum up the characteristics of stateful and stateless session beans as follows.

- Session beans are not shared: at any point in time, only one client can use a given bean.
- Session beans are not persistent: their state is not stored once they are no longer used by a client.
- Stateful beans should be used if the bean has to keep information about the client or about the interaction's history.

- Stateless beans should be used if each method provides a self-contained generic task, i.e. one which is not specific to any client nor requires any previous context.
- Stateless beans provide better scalability than stateful beans: typically, to support a given number of clients, the server needs fewer stateless beans than stateful ones because stateless beans can be reused among clients.
- Stateless beans provide better performance than stateful beans: calling a stateful bean method may require deserialising the bean from secondary storage.
- Beans usually use container-managed entity managers, although it is possible for beans to use application-managed entity managers in order to explicitly control the entity managers and the transactions.

For your reference, we conclude with a summary of the annotations we have used with session beans.

Table 6 Annotations for session beans

In package `javax.ejb`	
`@EJB`	The annotated variable is an injected session bean.
`@Local`	The annotated business interface will be accessed locally.
`@Remote`	The annotated business interface will be accessed remotely.
`@Remove`	The annotated method is the last to be executed in the session; after it completes, the stateful bean is removed from the container.
`@Stateful`	The annotated class defines a stateful session bean.
`@Stateless`	The annotated class defines a stateless session bean.
`@TransactionAttribute (TransactionAttributeType .value)`	Specifies the transactional requirements of the annotated session bean method, with *value* being `REQUIRED` or `NOT_SUPPORTED`. This annotation can also be used with a class, setting the default for all methods.
In package `javax.persistence`	
`@PersistenceContext( unitName = string, type = PersistenceContextType.value)`	The annotated variable is an injected container-managed entity manager for the given persistence unit. The *value* is either `EXTENDED` or `TRANSACTION`.

5.6 Transfer objects

The business interfaces we have shown so far require the client and server to communicate via individual attributes of type `String`, `int`, etc., to pass information back and forth. This can be quite cumbersome, error-prone and inflexible.

- It may be cumbersome if the client has to make multiple calls to get, piecemeal, all the information required (e.g. product description, product price, etc.).
- It may be error-prone if there is no simple way to check beforehand (i.e. on the client side) whether the information passed to the server is correct (e.g. the product ID).
- It may be inflexible if the server provides the information in a certain fixed way (e.g. as strings).

Further disadvantages are a heavier load on the network and poorer performance, because more remote calls and more checks are necessary.

SAQ 13

Show how some of these drawbacks are manifest in the code for The Music Store application in the previous sections.

ANSWER

The query methods return a pre-formatted string listing the models or instruments in the database. This is very inflexible: the client cannot control which information is presented to the user and how it is presented. This is an example of a badly designed business tier that addresses user interface issues, instead of just keeping to the business logic.

The client–server interaction is also error-prone, because the user is forced, by the `addInstrument` method, to type in each instrument ID, all of which have to be checked on the server side. If the user has mistyped an instrument ID, they have to type it again, leading to a further remote call to the server and a further check on the database.

To sum up: in defining the example business interfaces we have not followed the data granularity advice in Subsection 5.2. A much better way for clients and server to communicate is via **transfer objects** that contain all the necessary information in their instance variables. For example, it is simpler and more efficient to have a business interface like

```
public ProductTO getProduct(String id);
public void setCustomer(CustomerTO c);
```

instead of

```
public float getPrice(String id);
public String getDescription(String id);
public void setCustomer(String name, String address,
       String CreditCardCompany, String CCNumber, Date CCExpires);
```

where the `TO` suffix stands for 'transfer object'. It is simpler because fewer methods are required and their signatures are more concise; it is more efficient because fewer remote method calls have to be made.

One obvious solution is to use entities as transfer objects. After all, entities already encapsulate the information contained in the database and reflect the business concepts (product, customer, etc.) needed for the application. However, there are some reasons for using separate classes to define transfer objects.

- Entity classes reflect closely the way data is organised in the database. Such organisation is relevant for the server side of the application, but not for the client side. If clients are not shielded from the internal way data is structured, any changes to that structure will ripple to the client code.
- Related to the previous point is the fact that the way customers view information may differ from the way it is stored. For example, a customer might need to be presented with complete information about a product, including manufacturer details. However, such information may have been spread throughout multiple entities. Using a single transfer object is a way to aggregate it.
- Entity classes may contain information that is irrelevant or confidential to clients. For example, a product entity might contain details about when it was added to the stock, which is not of concern to the customer. Besides, sending such extra information to the client is a waste of network resources.

Transfer objects represent the client's view of data, while entities represent the server's view. Although there is necessarily an overlap between those views, they are usually distinct – hence the need for two different kinds of object. Entities are suitable for data

transfer between session beans and the database; transfer objects are suitable for data transfer between clients and session beans.

If the client is remote, which is the usual case, the transfer objects must be declared as `Serializable`, and they are copied in method calls and returns. This means that any change to the transfer object made by the client is not reflected in the server, and vice versa. Being serialised also means that any calls on the transfer object's methods will be executed locally by the receiver of the object – as happened in Subsection 2.4 with the call to `compute` on the `Factorial` object sent to the computation engine. Two examples illustrate how we can put these facts to practical use.

In the first example, the server wishes to send product information to the client for display purposes. The server just puts together the necessary details in a transfer object and sends it as the return value of a method. The object will never be returned by the client back to the server. In this case, the object may just contain public instance variables and no methods. The client is free to change the variables as it wishes, because it is working on a local copy, without any implication for the transfer object held by the server.

```
public class ProductTO implements Serializable
{
  public String id;
  public float price;
  public String description;
}
```

Having defined the transfer object, the business interface becomes simply as follows.

```
@javax.ejb.Remote
public interface Queries
{
  public ProductTO getProduct(String id);
  // other methods, but none with a ProductTO argument
}
```

In the second example, the client is about to update the customer's details (e.g. the home address). The server sends a transfer object with the current data about the customer; the client does the changes and returns the object. In this case, it is best for the object to have private instance variables and public getter and setter methods. The setter methods should include validation code (e.g. the address cannot be empty). This code will be executed locally on the client, not on the server, because the client has its own copy of the transfer object. This frees the session bean from having to check the new data once it gets back the transfer object. Furthermore, the object may contain the unique customer ID in a private variable, as long as no setter method is provided for it. In this way, the client cannot change the ID, but the server can read it to know which database record has to be updated.

```
public final class CustomerTO implements Serializable
{
  private String id, name, address;

  protected CustomerTO(String id)
  {
    this.id = id;
    setName("unknown");
    setAddress("unknown");
  }
```

```
    public void setAddress(String address)
                                    throws InvalidArgumentException
    {
      if (address == null || address.equals(""))
        throw new InvalidArgumentException("Address is empty");
      this.address = address;
    }

    // method setId is NOT provided
    // method setName does a similar check
    // methods getAddress, getId, getName specified as usual
}
```

Note that the class is `final` and the constructor is `protected`. In this way, only the server-side classes in the same package as `CustomerTO` can call the constructor, which cannot be overridden because the class cannot be extended. This guarantees that the unique customer ID is properly initialised on the server and is not then changed by the client. Having defined the transfer object, the bean's business interface can be written. It has two methods for a client to ask the server for a new or existing customer record, and a method for the client to send to the server any updates to a customer record. The `createCustomer` method creates a unique ID, passes it to the `CustomerTO` constructor and returns the created transfer object.

```
@javax.ejb.Remote
public interface CustomerRemote
{
  public CustomerTO createCustomer();
  public CustomerTO getCustomer(String id);
  public void updateCustomer(CustomerTO customer);
}
```

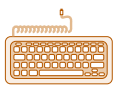
To sum up, we can explore the fact that each side has its own copy of the transfer object and calls its methods locally. In the first example, we made the transfer object as lightweight as possible; in the second we delegated validation from the server to the client.

SAQ 14

Summarise the rationale for using transfer objects.

ANSWER.....

Transfer objects aggregate data according to the client's view, thereby shielding entities from clients. Transfer objects simplify business interfaces and reduce network load, by reducing the number of methods and calls necessary to transfer data between clients and session beans.



Activity 7.5
Transfer objects.

The last activity for this unit defines a transfer object class for instruments and uses it in the application for The Music Store.

6 Summary

Remote Method Invocation (RMI) provides the ability for objects in one JVM to call methods on objects located in a different JVM. Although hidden by Java EE, RMI is an important subject of study in its own right because it raises a number of recurring techniques and issues that are typical of distributed systems, such as the implications of multiple object copies or the issue of locating objects and services by name. The RMI registry is a simple example of a naming service. More sophisticated directory services that locate services by hierarchical names or by attributes can be accessed via the JNDI API.

For RMI to work, it must be possible to send objects from one machine to the other, e.g. as arguments of the method calls. This is achieved through object serialisation, the facility for marshalling a complex data structure as a sequence of bits across a network and then unmarshalling it to reconstruct the sent object.

Java EE provides a persistence API for the business tier to access the data in the resource tier in an easier way than by making calls to JDBC. So-called entity classes can mirror, in an object-oriented way, the tables of the underlying relational database, and therefore make access to them more natural within the Java language. Entities represent individual records in the database and may thus be shared by many business activities. Entities are linked to records via persistence contexts. Changes to entities within a persistence context will be reflected in the database, while changes to detached entities, i.e. those outside a persistence context, are not. Entity managers are the programmatic interface of persistence contexts, allowing entities to be put into or removed from a persistence context.

As a further aid to the programmer, Java EE makes much use of annotations, a language feature available since Java 1.5 to reduce the need for writing repetitive code. Two examples are the ability to annotate an entity class with the name of the table where the data is located and to annotate which variable of the entity class corresponds to the primary key. The Java EE infrastructure will then automatically use the given table to retrieve and store instances of that class, and will make sure that the values of the primary key are indeed unique, without the developer having to program all this explicitly.

Entity classes represent only the data schema of the resource tier; they do not contain any code that processes the data. Such code is contained in session beans. Each session bean class defines a particular business activity, e.g. the purchase of a product. Stateful beans define activities that span multiple calls, while stateless beans implement one-off services. Each session bean services the requests of a single client, thereby retrieving and updating entities. Clients and session beans usually communicate data via transfer objects.

The Java EE infrastructure handles much of the concurrency and distribution issues automatically for the developer, favouring a style of ‘programming by default’. Nevertheless, it is important to be aware of what happens ‘under the hood’, so that the right design choices can be made; for example, which kind of entity manager or persistence context to use, when to synchronise methods, when to delegate data validation to clients, when to override the default transaction mechanism, etc.

LEARNING OUTCOMES

When you have completed your study of this unit you should be able to do the following:

- explain how Remote Method Invocation implements distributed objects and write simple distributed applications using it;
- explain why, and how, distributed systems use name and directory services;
- explain what annotations are and give some examples of what they are used for;
- explain the role of entity classes in making applications persistent and how entities are managed;
- explain what the role of session beans is, what types of session bean exist, and what their lifecycles are;
- explain what support EJB containers provide;
- extend the implementation of the business tier of simple applications, using any appropriate annotations, entity classes, transfer objects and session beans;
- discuss any trade-offs, and concurrency and distribution issues arising in the use of the above technologies.

Glossary

annotation A Java language construct that provides additional information about the class, method, variable, etc. that is being annotated. The information can be used at compile time or at runtime, typically to generate code, enforce additional constraints, or provide extra documentation. Each annotation has zero or more elements, each being a name–value pair.

application-managed entity manager An entity manager that has to be explicitly created and closed by the application.

bidirectional relation This kind of relation can be navigated in either direction – both tables/classes in the relation point to each other.

binding The association between a name (that must be unique within a certain context) and a resource (file, service, IP address, etc.). Name services store a set of bindings and allow users to create, change and remove bindings.

business interface An interface with the session bean methods that a client may call.

container-managed entity manager An entity manager that is implicitly created and closed by the Java EE infrastructure. By default, the entity manager automatically creates and commits transactions on method calls and returns.

dependency injection A technique whereby an object automatically obtains a reference to a resource it depends on. An example is the dependency injection of entity managers into session beans by the EJB container.

directionality (of a relation) The attribute of a relation that states whether it is unidirectional or bidirectional.

directory service An extension to a name service that allows resources to have attributes and to find resources that match given attribute values. LDAP is a widely used directory service protocol.

Domain Name Service (DNS) An example of a name service: it looks up internet domain names and returns the corresponding IP addresses.

Enterprise JavaBeans (EJB) The component model used by Java EE. There are three kinds of component: session beans and message-driven beans encapsulate business logic, servicing requests from clients; entities encapsulate the business data.

Enterprise JavaBeans (EJB) container The runtime infrastructure, usually part of a web server, that supports execution of EJBs.

entity An object that corresponds to persistent data in a relational database.

entity class A class that just holds data and may provide business logic, although the latter is usually delegated to session beans. Entity classes must be annotated with `@Entity`.

entity manager A runtime object that manages the lifecycle of entities, allowing them to enter or leave the persistence context, to be stored in or removed from the database.

export (a remote object) To make the remote object available so that it can accept incoming calls from clients.

extended persistence context A persistence context that has the same lifetime as an entity manager. This is the only possible setting for application-managed entity managers.

foreign key A column that uniquely refers to a record in another table.

inverse side In a bidirectional relation between entity classes, the class that is not the owning side.

Java Naming and Directory Interface (JNDI) An API to facilitate the uniform use of a variety of name and directory services from within Java programs.

location transparency The ability to manipulate remote objects in the same way as local ones.

many-to-many relation A relationship between two tables or entity classes, in which each record/entity of each table/class is related to zero or more records/entities of the other table/class. Usually, a many-to-many relation can be represented by two one-to-many relations with a third table/class.

many-to-one relation The inverse of a one-to-many relation.

metadata Data that describes other data. A relational schema is an example of metadata, and so are annotations like `@Table`.

multiplicity (of a relation) The attribute of a relation that states whether it is one-to-one, one-to-many, many-to-one or many-to-many.

name (or naming) service A facility that binds names to resources (web servers, printers, databases, etc.) so that clients can locate resources by name instead of having to know their exact location. The RMI registry is an example of a name service, with resources being Java objects.

navigation (of a relation) The ability to access the related record/entity of the given record/entity.

object serialisation The marshalling and unmarshalling of objects so that they can be sent as streams of bits across JVMs. A class whose instances may be serialised has to implement the `Serializable` interface.

one-to-many relation A relationship between two tables or entity classes, in which each record or entity of the first table/class is related to zero or more records/entities of the second table/class.

one-to-one relation A relationship between two tables or entity classes, in which each record/entity of each table/class is related to exactly one record/entity of the other table/class.

owning side The entity class that holds the foreign key in the underlying table. If both underlying tables involved in the relation have foreign keys, then either class can be the owning side.

persistence context A set of entities that are mapped to corresponding records in the database. Entities in the persistence context are in the Managed or Removed state, while those outside the context are in the New or Detached state.

persistence unit The entity classes, and the relational database that they are associated with.

primary key A column that uniquely identifies a record among all others in the same table.

proxy object An object that 'stands in' for another object. Calls to the proxy's methods are redirected to the referred object.

registry A naming service that allows names to be bound to object stubs for later lookup. Names have to be unique within a registry. Registries usually run in their own JVMs in the server host.

relational schema The structure, in terms of tables and columns, of the data in a relational database.

remote interface An interface that extends `Remote` and declares a set of remote methods.

remote method A method that can be called from another JVM. Remote methods may throw a `RemoteException`.

Remote Method Invocation (RMI) Java's approach to allow objects in one JVM to call methods on objects in another JVM.

remote object An object that implements a remote interface. The object may provide other methods that are not to be called remotely. Remote object classes usually extend `UnicastRemoteObject`.

remote reference See **stub object**.

security manager A runtime service that checks whether the defined security policy is being followed.

security policy A policy that defines the permissions of code that is downloaded at runtime.

session bean A type of EJB that handles an interactive session between a client and the application server. Session beans are neither persistent nor shared.

stateful session bean A session bean that implements a single business service that spans multiple method calls, hence requiring state to be kept between calls.

stateless session bean A session bean that does not keep state between calls, because each method implements a separate service: each session consists of a single method call.

stub object A proxy for a remote object. There is a copy of the stub object on the registry and on each client JVM that wants to use the corresponding remote object.

transaction-scoped persistence context A persistence context that has the same lifetime as a transaction. This is the default setting for persistence contexts used with container-managed entity managers.

transfer object An object used for transferring aggregated data between clients and session beans.

unicast communication Point-to-point communication.

unidirectional relation This kind of relation can be navigated in only one direction, namely from the table/class that holds the foreign key to the other table/class.

References

Liu, M.L. (2004) *Distributed Computing: Principles and Applications*, Addison-Wesley.

Index